Data Structure and Algorithm Design

ME/MSc (Computer) – Pokhara University

Prepared by:

Assoc. Prof. Madan Kadariya (NCIT)

Contact: madan.kadariya@ncit.edu.np

Chapter 1:
Introduction to Basic Data Structures and Algorithm Design (6 hrs)

Outline

1. Review of Data Structure and Algorithm
2. Basic Data Structures
   - Arrays, Stack, Queue, Linked list, Tree and Graph
3. Algorithm Design Approaches
   - Divide and Conquer, Greedy Algorithm, Backtracking Algorithms, Branch & Bound and Randomized Algorithms
4. Algorithm Analysis
   - Time and Space Complexity
   - Asymptotic notations
     - Big O, Big Theta and Big Omega

Review of Data Structure and Algorithm

- **Data structure** is a representation of logical relationship between individual elements of data.
- A **DS** is a way of **organizing** the data items, **their relationship** and **operations** on those data.
- **Algorithm** is used to describe the way data is processed(step-by-step finite sequence of instruction to solve a well-defined computational problem).

  Algorithm + Data structure = Program
- Optimization of a program is directly concerned with algorithm design.
- The data structure and the operation on organized data items can integrally solve the problem using computer

  Data structure = Organized data + Operations
- In short **DS** is:
  - How to store a collection of objects in memory?
  - What operations we can perform on that data?
  - The algorithms for those operations?
  - How time and space efficient those algorithms are?

Review of Data Structure and Algorithm

- A data structure is said to be **Linear***(logical and/or memory structure*) if its elements form a sequence or a linear list.
  - Array, Stacks, Queues, Linked lists (static and logical structure) .
- A data structure is said to be ***Non-linear****(logical and/or memory structure*) if its elements form a non-linear (like hierarchical classification) where, data items appear at various levels.
  - Trees, Graphs (*logical and memory*)

- Data structure can be classified into 2 categories
- **Primitive data structures**: Basic data structures and are directly operated upon by the machine instructions, which is in a primitive level. Eg: integers, floating point numbers, characters, string constants, pointers etc.

- **Non-primitive data structures**: It is a more sophisticated data structure emphasizing on structuring of a group of homogeneous (same type) or heterogeneous (different type) data items. Eg: Array, List, files, Linked list, Trees, Graphs etc.

Review of Data Structure and Algorithm

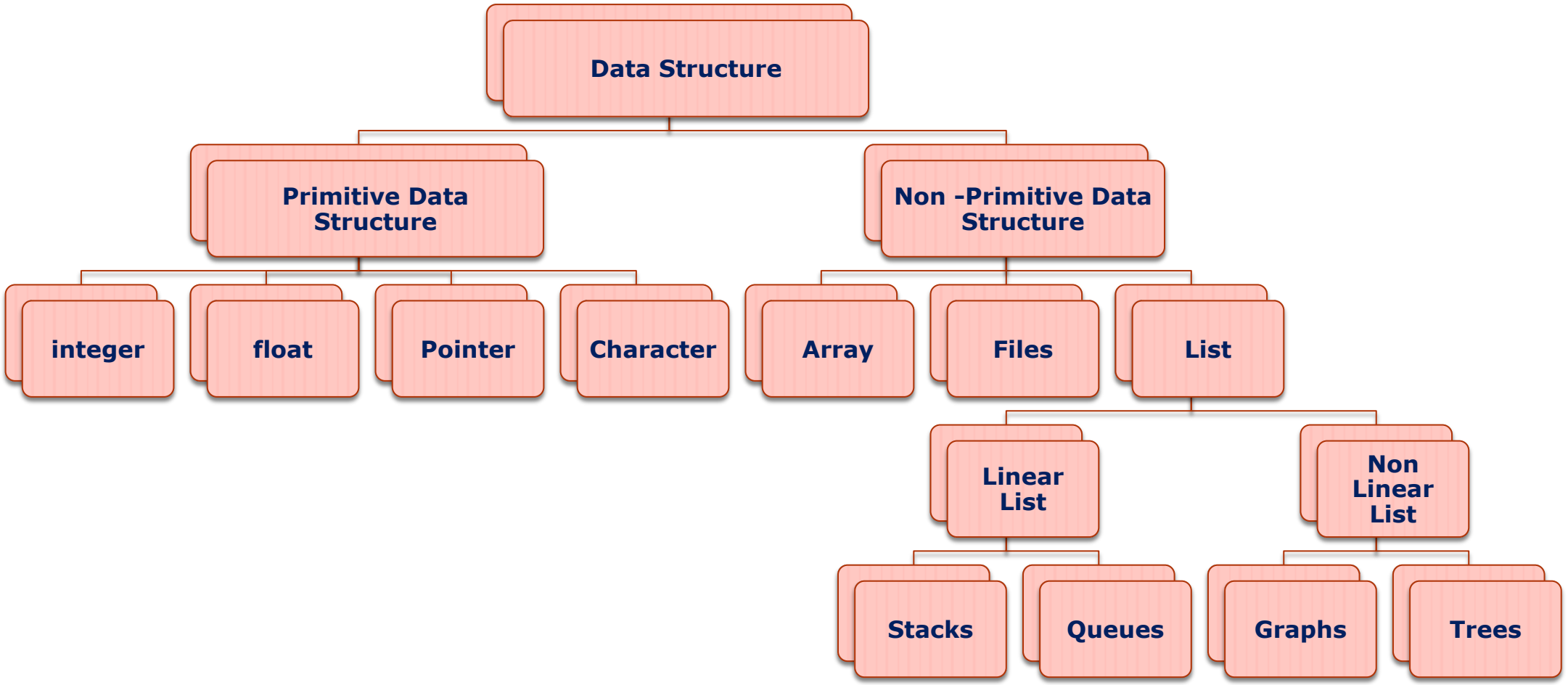


Fig: Classification of Data Structure

Review of Data Structure and Algorithm

Data Types

- A **data type** is a classification of data, which can store specific type of information.
- Data types are primarily used in computer programming in which variables are created to store data.
- Each variable is assigned a data type that determine what type of data the variable may contain.
- Data type is a collection of values and set of operation on those values.
- **Example**: in the statement int a=5, 'a' is a variable of data type integer which stores integer value 5 and allow different mathematical operations like addition, subtraction, multiplication and division.

Abstract Data Types (ADT)

- We can perform some specific operations with data structure, so data structure with these operations are called **Abstract Data Types (ADT).**
  - ADT have their own data type and methods to manipulate data.
  - It is a useful tool for specifying the logical properties of a data type.
  - A data type or data structure is the implementation of ADT.

Review of Data Structure and Algorithm

Abstract Data Types (ADT)

- ADT allow us to work with abstract idea behind a data type or data structure without getting bagged down in the implementation detail. The abstraction in this case is called an abstract data type.

Advantages of ADTs

- **Encapsulation**
  - Encapsulation is a process by which we can combine code and data and it manipulates into a single unit.
  - While working with ADT, the user doesn't require knowing how the implementation works because implementation is encapsulated in a simple interface.
- **Flexibility**
  - If different implementations of ADT are equivalent then they can be interchangeable in the code. So user have flexibility to select the one which is most efficient in different situations.

Review of Data Structure and Algorithm

Data Structure Operations (Basic Primitive Operations)

- **Traversing**:-Accessing each record so that some item in the record may be processed.
- **Searching** :- Finding the location of the record with the given key value
- **Inserting** :- Adding new record to the data structure
- **Deleting** :- Removing record from the data structure
- **Sorting** :- Arranging the record in some logical order
- **Merging** :- Combining the record of different sorted file into single sorted file.

Example: An array A having n element is ADT which have following operations

- **Create(A)** :- create an array A
- **Insert(A,x)** :- insert an element x into an array A in any location
- **Delete(A,x)** :- remove element x
- **Traverse(A)**:- access each element of an array A
- **Modify(A,x,y)** :- Change the old element x with new element y
- **Search(A,x)** :- search element x in an array A
- **Merge (A,B,C)** :- Combine the elements of array B and C and store in new array A

Basic Data Structure (Outline)

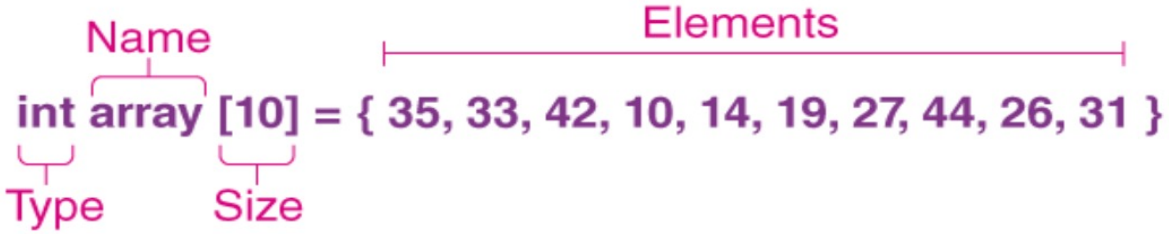
1. Arrays
2. Stack
3. Queue
4. Linked list
5. Tree
6. Graph

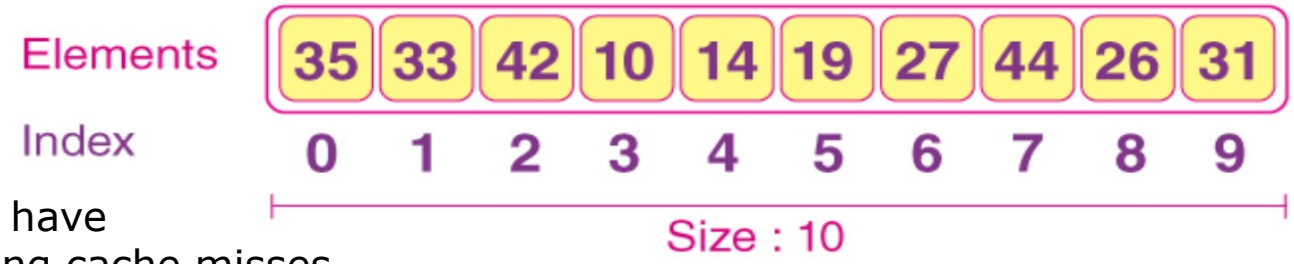
Basic Data Structure (Array)

Basic Data Structure (Array)

Array

- A linear data structure storing elements in contiguous memory locations. Each element is accessible by an index.





Advantages:

- **Constant time** (O(1)) access to elements via indexing.
- **Memory efficiency** for small data (no extra pointers).
- **Cache-Friendly:** Due to contiguous allocation, arrays have better cache locality, improving performance by reducing cache misses.

Disadvantages:

- Fixed size; resizing is costly.
- Insertion and deletion are inefficient (O(n))- **Costly**
- **Wasteful of Memory:** If not fully utilized or if allocated with excess capacity to accommodate future growth.

Types of Array

- Single Dimension Array (with 1 subscript)
- Two Dimension Array (with 2 subscript)
- Multi Dimension Array (With Multiple subscript)

Applications:

- Storing and manipulating tabular data.
- Used in matrix operations and image processing.
- **Implementing Other Data Structures:** The underlying structure for more complex data structures like heaps, hash tables, and dynamic arrays.

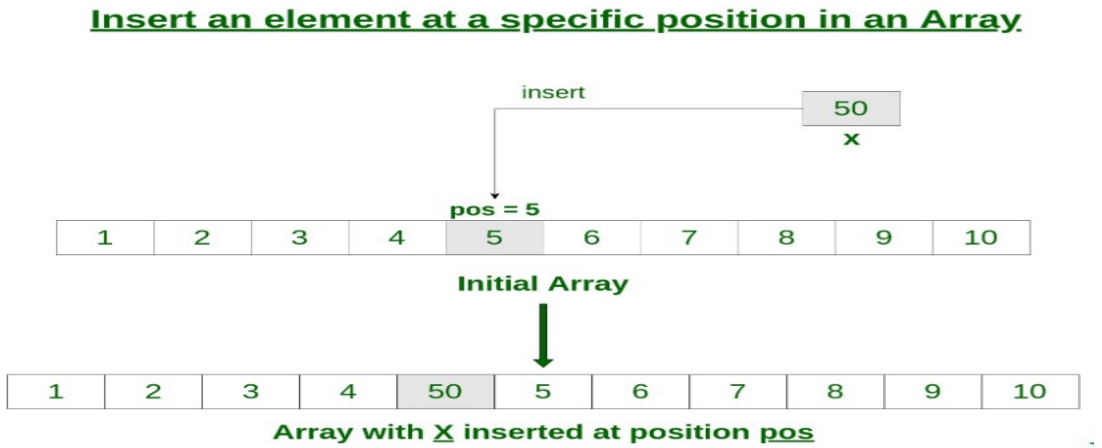
Basic Data Structure (Array)

Insertion into an Array

At a specified position in an existing array

- Write over the current contents at the given index (which migl not be appropriate), or
- The item originally at the given index must be moved up one position, and all the items after that index must ***shifted up.***

```
for (int pos = numItems; pos >= index; pos--) {
        items[pos] = items[pos-1];
    } // end for
  items[index-1] = item;
    numItems++;
```

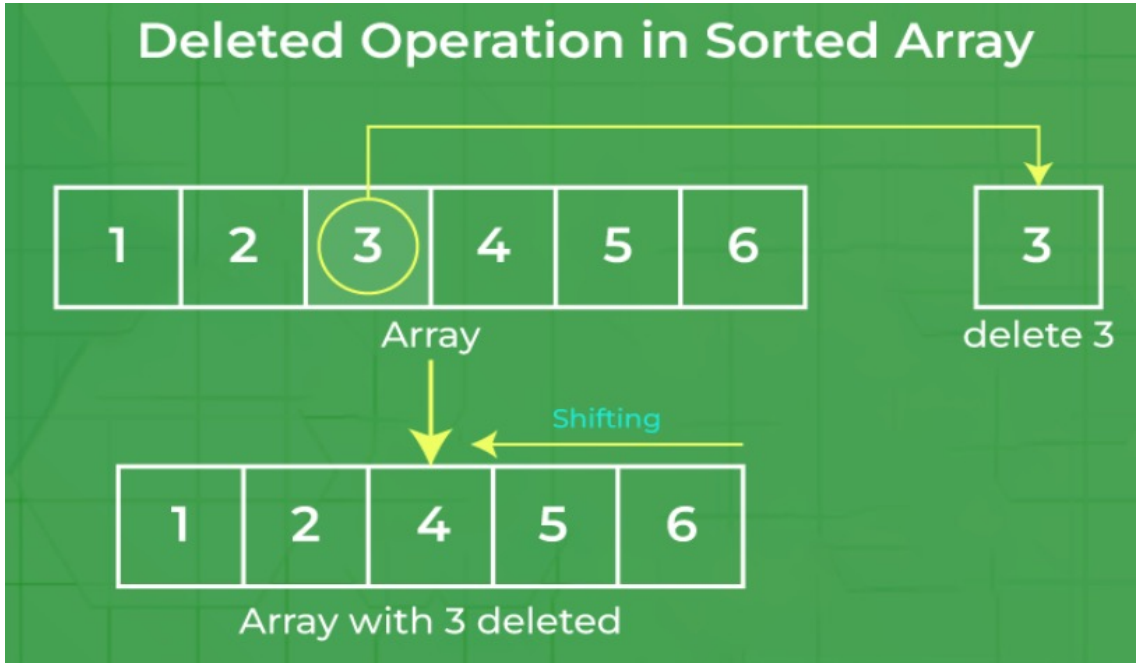


Deletion from an Array

From a specified position of an existing array

- Leave ***gaps*** in the array, i.e. indices that contain no elements, which in practice, means that the array element has to be given a special value to indicate that it is "empty", or
- All the items after the (removed item's) index must be ***shifted down***

```
for (int pos = index+1; pos <= size(); pos++) {
      items[pos-2] = items[pos-1];
    }  // end for
    numItems--;
```



Basic Data Structure (Stack)

Basic Data Structure (Stack)

Stack

- It operates on the principle of "**Last-In, First-Out**" (LIFO), order.
- Access is allowed only at one point of the structure, normally termed the ***top*** of the stack
  - access to the most recently added item only

Key Operations

- **Push:** Adds an element to the top of the stack.
- **Pop:** Removes the top element from the stack and returns it. **Peek (or Top):** Returns the top element of the stack without removing it.
- **IsEmpty:** Checks whether the stack is empty.

Characteristics

- **Dynamic Size:** Most stack implementations grow and shrink dynamically with the addition and removal of elements.
- **Fast Operations:** Adding and removing elements from a stack is generally fast, with operations typically having a constant time complexity, O(1).

Advantages

1.Efficient memory use for nested operations (e.g., recursion).
2.Easy implementation for limited operations.

Disadvantages

1.Fixed size in static stacks.
2.Access limited to the top of the stack.

Applications

- **Function Call Management**: When a function is called, its execution context is pushed onto a call stack, and when the function returns, its context is popped off.
- **Undo Mechanisms**: Each action is pushed onto a stack, and undoing an action pops it off the stack and reverses it.
- **Syntax Parsing**: Compilers and interpreters use stacks for parsing and evaluating expressions or checking the syntax of code with nested structures like parentheses.
- **Backtracking**: In algorithms that involve searching or traversing through possible solutions (like maze solving), stacks are used to keep track of the path taken and backtrack if necessary.
- Expression Evaluation and Conversion

Implementation

- **Array-Based Implementation:** Using an array
- **Linked List Implementation:** Using Linked List

Basic Data Structure (Stack)

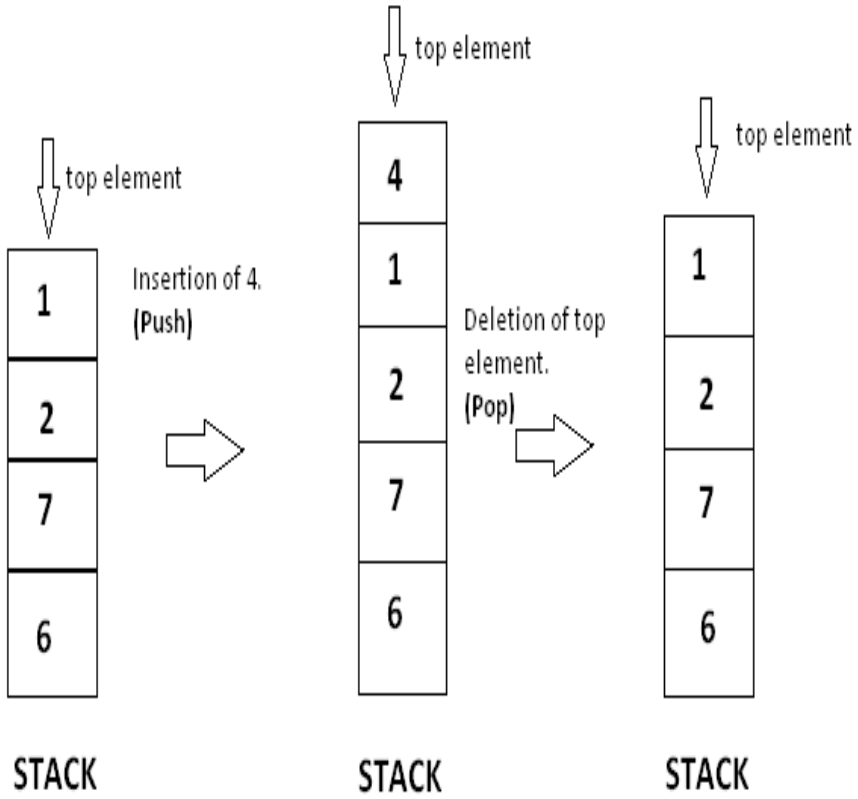
Static Implementation (Using Array)

Algorithm for Push

1. If TOP = MAX – 1, then:
   1.1. Display "The stack is in overflow condition"
   1.2. return/exit
2. TOP = TOP + 1
3. STACK [TOP] = ITEM
4. return/exit

Algorithm for POP

1. If TOP < 0, then
1.1. Display "The Stack is empty"
1.2. return/exit
2. Else remove the Top most element
3. DATA = STACK[TOP]
4. TOP = TOP – 1
5. return/exit



Time Complexity

- Push: **O(1)**
- Pop: **O(1)**
- Peek: **O(1)**
- IsEmpty: **O(1)**

Space complexity

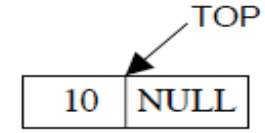
- Depends on the number of elements it contains, typically **O(n)** for storing n elements.

Basic Data Structure (Stack)

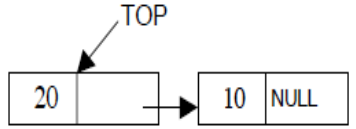
Dynamic Implementation (Linked List)

Algorithm for Push

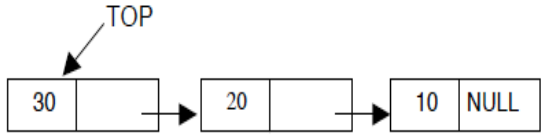
1. Input the DATA to be pushed
2. Create a NewNode
3. NewNode → data = DATA
4. NewNode → Next = TOP
5. TOP = NewNode
6. Exit/Return



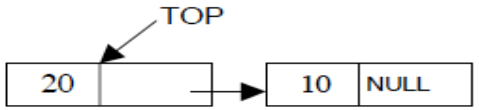
Push (10)



Push (20



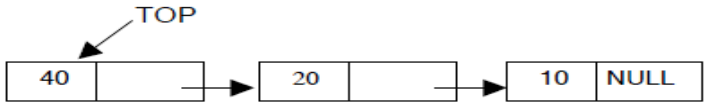
Push (30)



Pop (i.e. X=30)

Algorithm for Pop

1. if (TOP is equal to NULL)
   1.1. Display "The stack is empty"
2. Else
   2.1. TEMP = TOP
   2.2. Display "The popped element TOP → DATA"
   2.3. TOP = TOP → Next
   2.4. TEMP → Next = NULL
   2.5. Free the TEMP node
3. Exit/Return



Push (40)

Summary of Complexity

Operation	Best Case	Worst Case (with resizing)	Amortized
Push	$O(1)$	$O(n)$	$O(1)$
Pop	$O(1)$	$O(n)$	$O(1)$
Peek	$O(1)$	$O(1)$	$O(1)$
IsEmpty	$O(1)$	$O(1)$	$O(1)$

Note: **Amortized complexity** is the total expense per operation, evaluated over a sequence of operations.

Basic Data Structure (Queue)

Basic Data Structure (Queue)

- A linear data structure that follows **First In, First Out (FIFO)** principle.
- Insertion is done through **rear** and deletion is done through **front.**

Key Operations

- **Enqueue:** Adds an element to the end of the queue.
- **Dequeue:** Removes the element at the front of the queue and returns it.
- **Peek (or Front):** Returns the element at the front of the queue without removing it
- **IsEmpty:** Checks whether the queue is empty.

Characteristics

- **Dynamic Size:** Most queue implementations grow and shrink dynamically with the addition and removal of elements.
- **Efficient Operations:** Enqueue and dequeue operations are generally fast, with operations typically having a constant time complexity.

Advantages

1. Ideal for sequential processing.
2. Efficient in managing resource sharing.

Disadvantages

1. Fixed size for static queues.
2. Inefficient memory utilization in simple queues.

Applications:

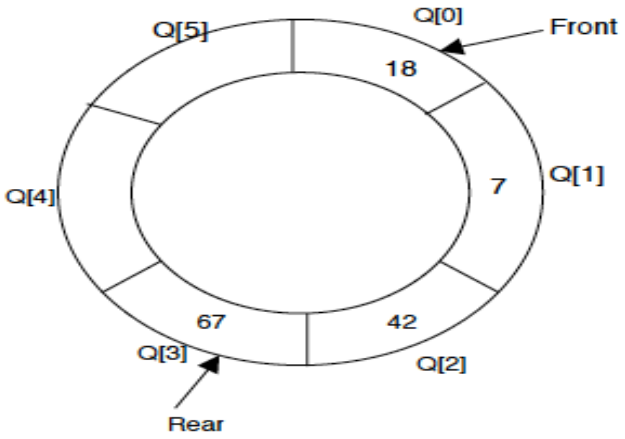
1. Print job scheduling.
2. **Process Scheduling**: Task scheduling in operating systems.
3. **Handling Requests:** Web servers use queues to handle incoming request traffic.
4. **Data Streaming:** Queues can be used to buffer data chunks before they are processed, ensuring that they are dealt with in the correct order.
5. **Breadth-First Search (BFS):** In graph algorithms, queues are used to traverse the graph level by level, starting from a given node.

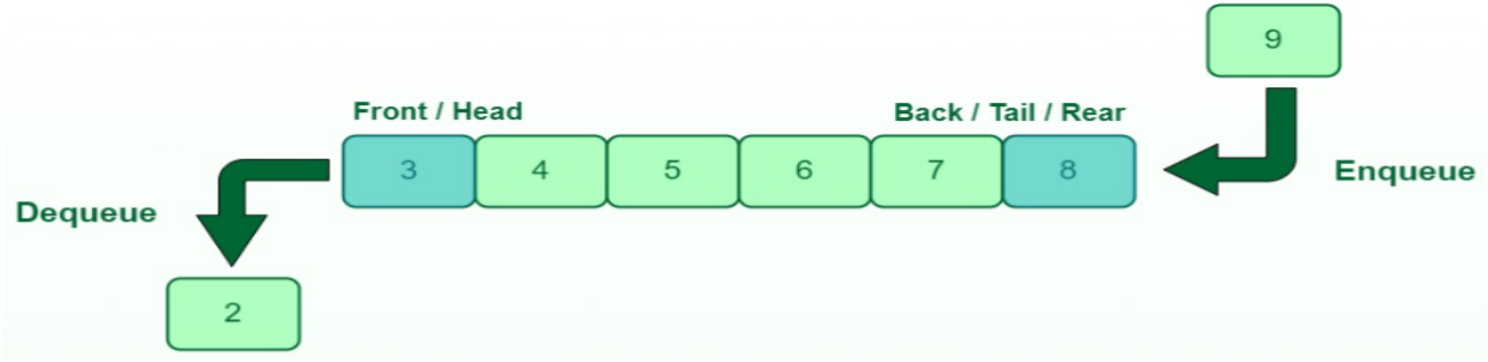
Implementation

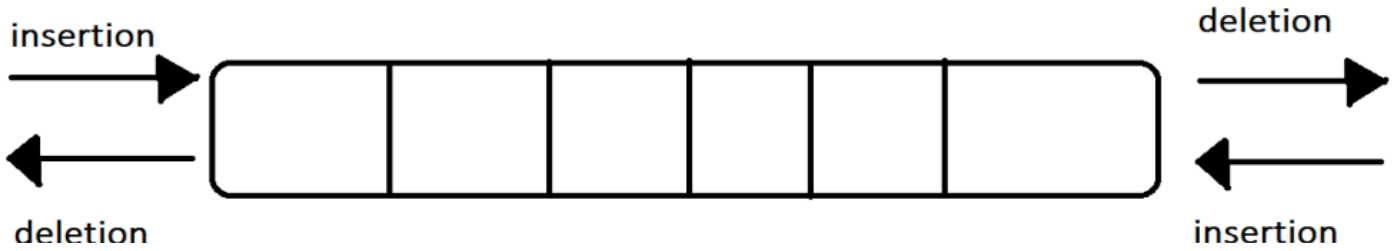
- **Array-Based Implementation:** Using an array
- **Linked List Implementation:** Using Linked List

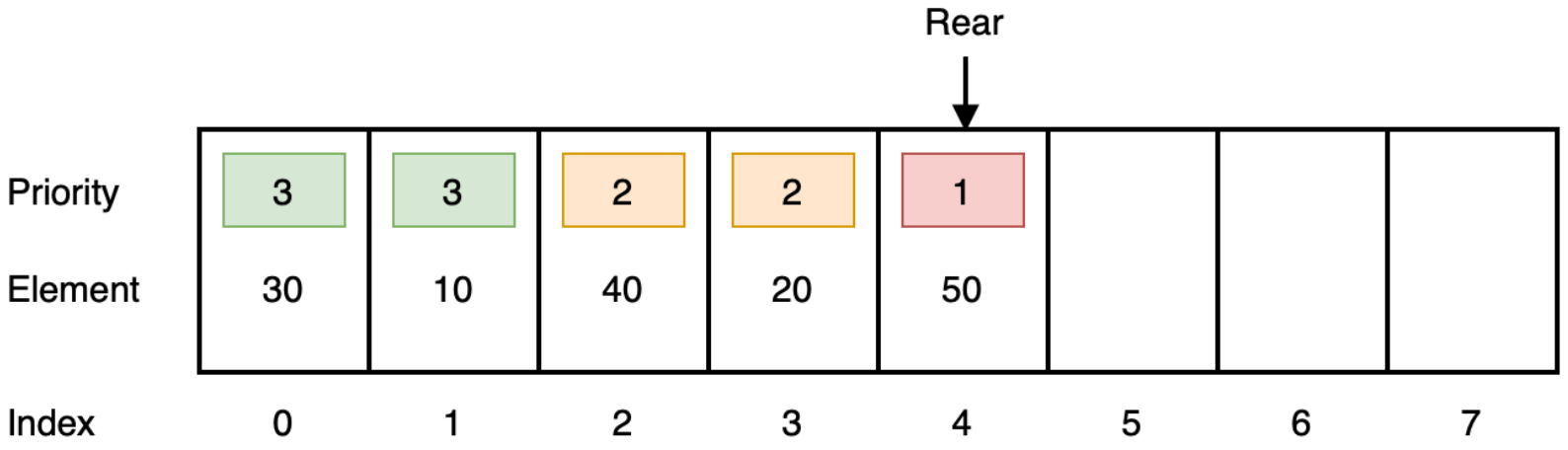
Basic Data Structure (Queue Types)

1. Linear Queue
2. Circular Queue
3. Double Ended Queue
4. Priority Queue









Basic Data Structure(Queue Implementation)

Static Implementation of Linear Queue

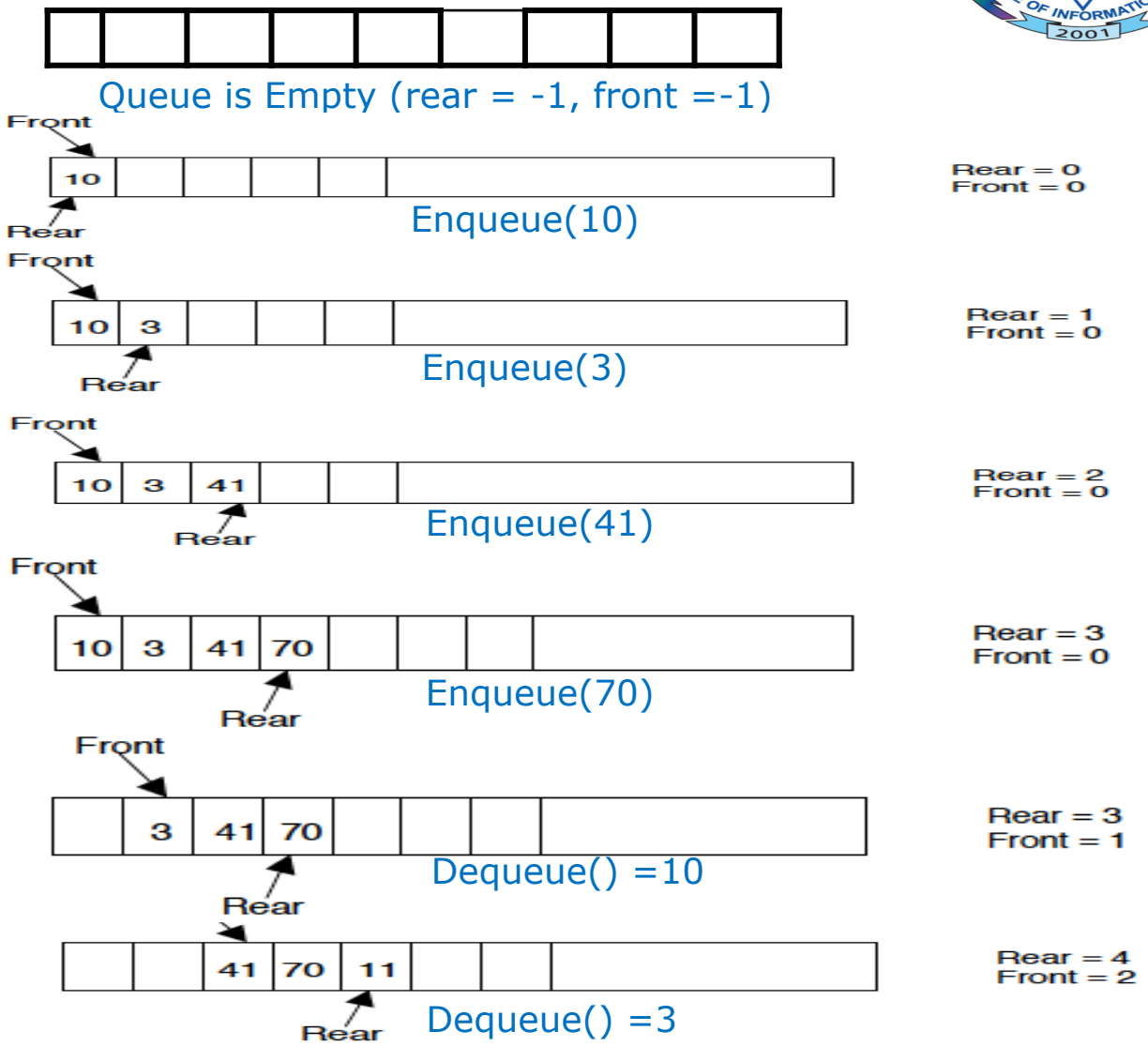
Algorithm for Enqueue

1. Input the value to be inserted and assign to variable "data"
2. If (rear >= MAX-1)
   2.1. Display "Queue overflow"
   2.2.. Return/Exit
3. Else
   3.1. If (front is equal to -1 and rear is equal to -1) [First time insertion]
      3.1.1. front = 0
4. rear = rear +1
5. Q [rear] = data
6. Return/Exit

Algorithm for Dequeue

1. If (rear< front) or if (front ==-1 && rear == -1);[checking for queue is empty]
   i. front = -1, rear = –1;(optional)
   ii. Display "The queue is empty"
   iii. Return/Exit
2. Else
   2.1. Data = Q [front]
3. front = front +1
4. Return/Exit

Overflow Condition: If Rear = MAX-1
Underflow Condition: If front = -1 and rear = -1 (Initial Condition) or rear < front
One Element: If rear = front.
Number of Elements present in a Queue: rear – front +1



Basic Data Structure(Circular Queue Implementation)

Static Implementation of Circular Queue

Algorithm for Enqueue

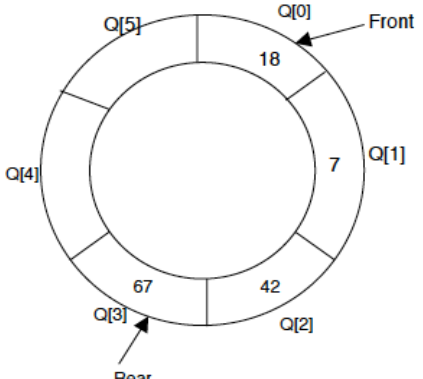
1. If ((FRONT is equal to 0 && rear = MAX-1) OR front = rear+1) //check for queue full
   - 1.1. Display "Queue is full"
   - 1.2. Return/Exit
2. If (FRONT is equal to – 1 && rear == -1) //Empty Queue (inserting first time)
   - 2.1. FRONT = 0
   - 2.2. REAR = 0
3. Else
   - 3.1. REAR = (REAR + 1) % SIZE
4. Input the value to be inserted and assign to variable "DATA"
5. Q [REAR] = DATA
6. Exit/Return

Algorithm for Dequeue

1. If (FRONT is equal to – 1 and rear == -1)
   - 1.1. Display "Queue is empty"
   - 1.2 Return/Exit
2. Else
   - 2.1. DATA = Q [FRONT]
3. If (REAR is equal to FRONT) //Queue has only one element
   - 3.1.FRONT = –1
   - 3.2. REAR = –1
4. Else
   - 4.1. FRONT = (FRONT +1) % SIZE
5. Return/Exit

Algorithm for Traverse

1. Initialize front_pos = FRONT and rear_pos = REAR
2. If front_pos is equal to -1 and rear_pos is equal to -1
   - 2.1 Display queue empty
   - 2.2 Exit/return
3. If front_pos is less than or equal to rear_pos [Like linear array]
   - 3.1. Print Q[front_pos]
   - 3.2. front_pos = front_pos+1
4. Else [eg: rear = 2 and front = 5 where MAX = 10]
   - 4.1.Repeat until front_pos is less than or equal to MAX-1 [print upto MAX-1]
     - 4.1.1.Print Q[front_pos]
     - 4.1.2.Front_pos = front_pos+1
   - 4.2. Set front_pos = 0
   - 4.3. Repeat until front_pos is less than or equal to rear_pos [print upto rear]
     - 4.3.1. Print Q[front_pos]
5. Exit/Return



Time Complexity

- Enqueue: **O(1)**
- Dequeue: **O(1)**
- Peek: **O(1)**
- IsEmpty: **O(1)**

Space complexity

O(n)

Basic Data Structure (Linked List)

Basic Data Structure (Linked List)

- Linked List is a linear data structure where elements (nodes) are linked using pointers.
- Linked list allows for efficient insertion and removal of elements from any position in the sequence, making linked lists a versatile alternative to arrays in certain situations.

Key Operations

- **Insertion:** Insert a new node (at the beginning, at the end, at the specified position). Insertion time complexity is **O(1)** if you have direct access to the point of insertion.
- **Deletion:** Remove a node from the list (from the beginning, from the end, at the specified position).Deletion has a time complexity of **O(1)** under similar conditions as insertion.
- **Traversal:** To access or find a node. This operation has a time complexity of **O(n)**, where n is the number of nodes in the list.

Features/Advantages

- **Dynamic Size:** The size of a linked list can grow or shrink dynamically.
- **Memory Efficient:** Linked lists can be more memory efficient than arrays for certain applications because they do not reserve more memory than they need.
- **Use in Implementing Other Data Structures:** Linked lists are used as for implementing other complex data structures like stacks, queues, and even some types of hash tables.
- **Polynomial Arithmetic:** Linked lists are ideal for representing polynomials because they can efficiently store and manipulate terms with non-zero coefficients.
- **Garbage Collection Algorithms:** Some garbage collection algorithms use linked lists to keep track of free memory blocks or to implement algorithms such as mark-and-sweep.

Limitations/Disadvantages

- **Random Access:** Linked lists do not support efficient random access to elements.
- **Memory Overhead:** Each node in a linked list requires extra memory for storing the pointer(s) alongside the actual data.

Applications

- Implementation of stacks, queues, and graphs.
- Dynamic memory allocation.
- **Example**: Managing browser history.

Basic Data Structure (Linked List)

Types of Linked List

1. Singly Linked List
2. Doubly linked List
3. Circular Linked List
   - 3.1. Singly Circular Linked List
   - 3.2. Doubly Circular Linked List

Representation of Linked List

Singly Linked List

```
struct Node{
   int data;
   struct Node *next;
};
typedef struct Node *NODE;
```

Doubly Linked List

```
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
typedef struct Node *NODE;
```

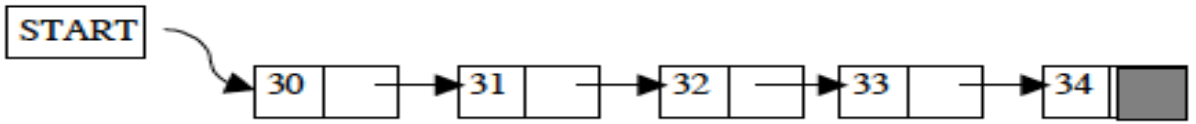


Fig: Singly Linked List

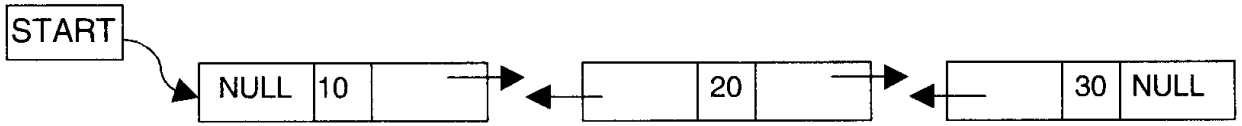


Fig: Doubly Linked List

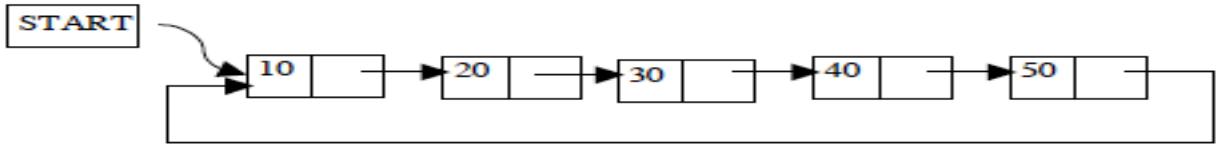


Fig: Singly Circular Linked List

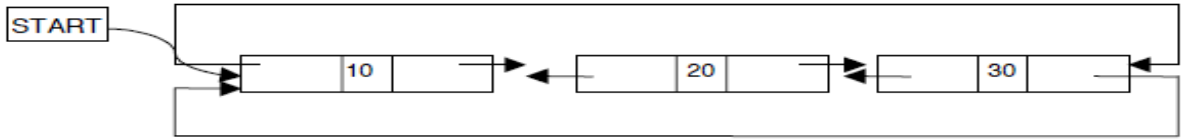
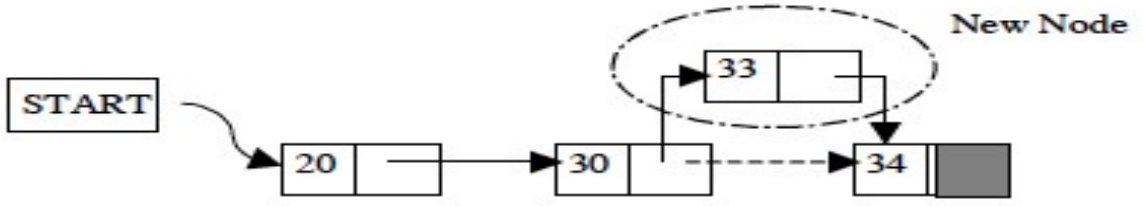


Fig: Doubly Circular Linked List

Basic Data Structure (Singly Linked List)

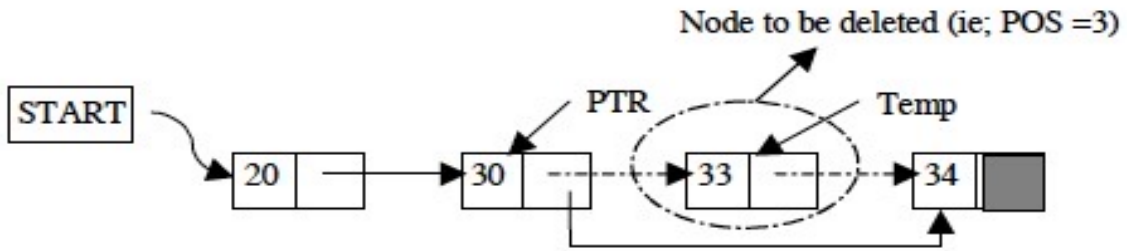
Algorithm for Inserting a Node at any specified position of Singly Linked List

1. Input DATA and POS to be inserted
2. Initialize TEMP = START; and i = 1
3. Repeat the step 3 until (i is less than POS-1)
   - 3.1. TEMP = TEMP->next
   - 3.2. If (TEMP is equal to NULL)
     - 3.2.1. Display "Node in the list less than the position"
     - 3.2.2. Exit/Return
   - 3.3. i = i + 1
4. Create a New Node
5. NewNode → DATA = DATA
6. NewNode → Next = TEMP → next
7. TEMP → next = NewNode
8. Return/Exit



Algorithm for Deleting a Node from specified position of Singly Linked List

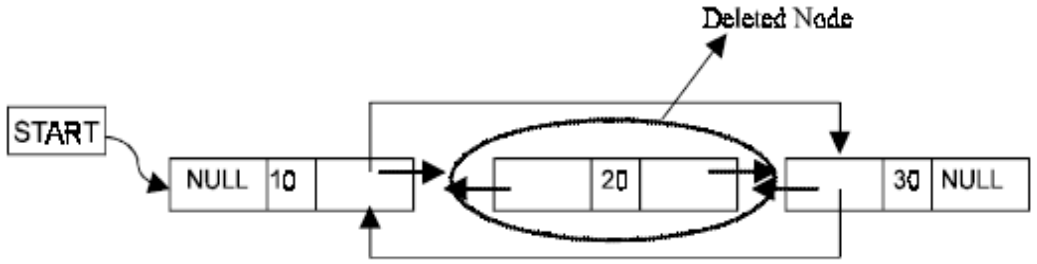
1. Input POS of node to be deleted
2. Initialize PTR = START; and i = 1
3. Repeat the step 3 until (i is less than POS-1)
   - 3.1. PTR = PTR->next
   - 3.2. If (PTR is equal to NULL)
     - 3.2.1. Display "Node in the list less than the position"
     - 3.2.2. Exit/Return
   - 3.3. i = i + 1
4. Initialize TEMP= PTR->next
5. PTR->next = TEMP->next (PTR->next->next)
6. Free the node, TEMP
7. Return/Exit

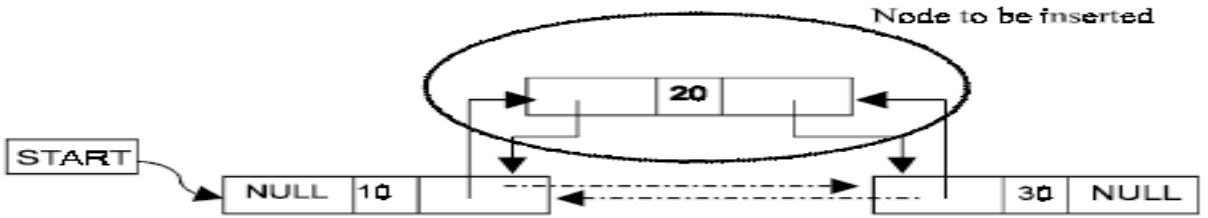


Basic Data Structure (Doubly Linked List)

Algorithm for Inserting a Node at any specified position of Doubly Linked List

1. Input DATA and POS to be inserted
2. Initialize TEMP = START; and i = 1
3. Repeat the step 3 until (i is less than POS-1)
   - 3.1. TEMP = TEMP → next
   - 3.2. If (TEMP is equal to NULL)
     - 3.2.1. Display "Node in the list less than the position"
     - 3.2.2. Exit/Return
   - 3.3. i = i + 1
4. Create a New Node
5. NewNode → DATA = DATA
6. NewNode → Next = TEMP → Next
7. NewNode → Prev = TEMP
8. (TEMP → Next) → Prev = NewNode
9. TEMP → Next = New Node
10. Return/Exit





Algorithm for Deleting a Node from specified position of Doubly Linked List

1. Input POS of node to be deleted
2. Initialize PTR = START; and i = 1
3. Repeat the step 3 until (i is less than POS-1)
   - 3.1. PTR = PTR → next
   - 3.2. If (PTR is equal to NULL)
     - 3.2.1. Display "Node in the list less than the position"
     - 3.2.2. Exit/Return
   - 3.3. i = i + 1
4. Initialize TEMP= PTR → next
5. PTR → next = TEMP → next (PTR → next → next)
6. (TEMP → next) → prev = PTR
7. Free the node, TEMP
8. Return/Exit

Basic Data Structure (Linked List)

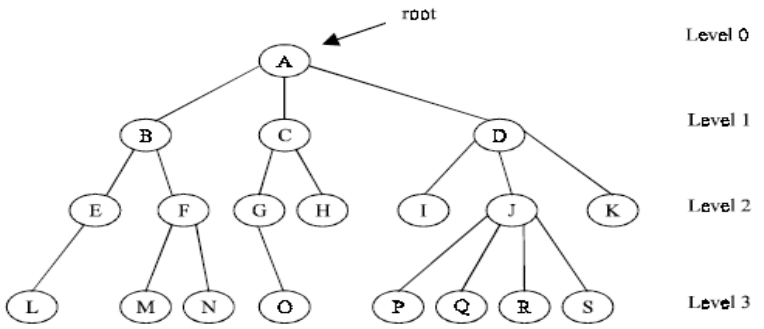
Complexity Comparison of SLL vs DLL

Feature	Singly Linked List	Doubly Linked List	Description
Insertion/Deletion at Head	O(1)	O(1)	At head or tail: Update pointers efficiently. At middle: Requires traversal.
Insertion/Deletion at Tail	O(n)	O(1)	**DLL**: at head or tail: Update pointers efficiently. **SLL** :at tail or middle: Requires traversal. At middle: Requires traversal.
Traversal (Forward)	O(n)	O(n)	Traverse sequentially to reach a node.
Traversal (Backward)	Not Supported	O(n)	**DLL**: Can traverse in both forward and backward directions.
Space Overhead	Lower (1 pointer per node)	Higher (2 pointers per node)	

Basic Data Structure (Tree)

Basic Data Structure (Tree)

- A tree is an ideal data structure for representing hierarchical data. A tree can be theoretically defined as a finite set of one or more data items (or nodes) such that:
  - There is a special node called the **root** of the tree.
  - Removing nodes (or data item) are partitioned into number of mutually exclusive (i.e., disjoined) subsets each of which is itself a tree, are called sub tree.



Basic Terminology

- **Root:** Root is the first node in the hierarchical arrangement of the data items.
- **Degree of a node** is the number of sub trees of a node in a given tree. Degree of node A=3, B=2, C=2 and D=3
- The **degree of a tree** is the maximum degree of node in a given tree. The degree of the above tree is 4.
- A node with degree 0 is called a **terminal node** or a **leaf**. M, N, I, O etc. are leaf node (terminal node).
- Any Node whose degree is not zero is called **non-terminal** node.
- A tree is structured in different **levels**. The entire tree is leveled in such a way that the root node is always of level 0.
- If a node is at level n, then its children will be at level n+1.
- **Path and Path length** : suppose $m_1, m_2, \ldots, m_k$ is a sequence of nodes in a tree such that m_1 is the parent of m_2, which is the parent of m_3, and so on, down to m_{k-1}, which is the parent of m_k. Then $m_1, m_2, \ldots, m_k$ is called a ***path*** from m1 to m_k in the tree. The ***path length*** or length of the path is $k-1$, one less than the number of nodes on the path.

Basic Data Structure (Tree)

Binary Tree

- A binary tree is either the empty set or a triple **T = (x, L, R),** where x is a node and L and R are disjoint binary trees, neither of which contains x.
- The node **x** is called the root of the tree T, and the subtrees L and R are called the *left subtree* and the *right subtree* of T rooted at x.
- A binary tree is said to be ***full*** if all its leaves are at the same level and every interior node has two children.
- The full binary tree of height *h* has $l = 2^h$ leaves and $m = 2^h - 1$ internal nodes.
- The full binary tree of height h has a total of $n = 2^{h+1} - 1$ nodes.
- The full binary tree with n nodes has height $h = lg(n+1) - 1$

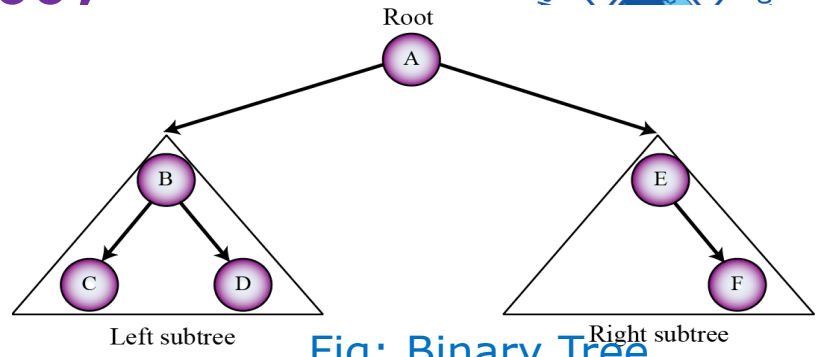


Fig: Binary Tree

- A ***complete binary tree*** is either a *full binary tree* or one that is full except for a segment of missing leaves on the right side of the bottom level.
- In a complete binary tree of height h,

  $h + 1 \le n \le 2^{h+1} - 1$ and h = ⌊ lg n ⌋
- Complete binary trees are important because they have a simple and natural implementation using ordinary arrays.
- The *natural mapping* is actually defined for any binary tree: Assign the number 1 to the root; for any node, if *i* is its number, then assign 2*i* to its left child and 2*i*+1 to its right child (if they exist).
- This assigns a unique positive integer to each node. Then simply store the element at node i in a[i], where a[] is an array.

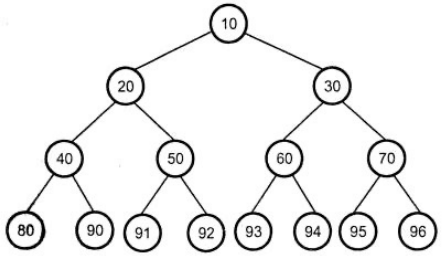


Fig: Full Binary Tree

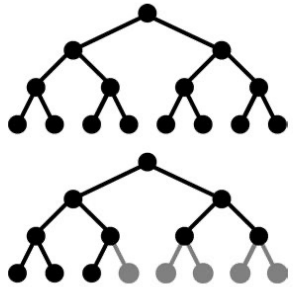


Fig: Full and complete Binary Tree

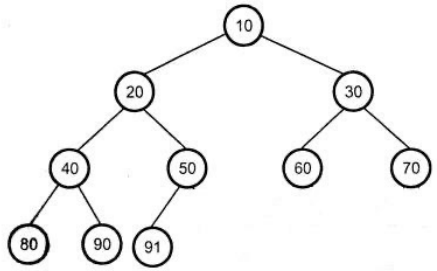


Fig: Complete Binary Tree

Basic Data Structure (Tree)

Binary Tree Representation

1. Array Representation

- If a complete binary tree with n nodes (depth = log n + 1) is represented sequentially, then for any node with index i, $1 \le i \le n$, we have:
- *parent*(i) is at $i/2$ if i!=1. If i=1, i is at the root and has no parent.
- *leftChild*(i) is at $2i$ if $2i \le n$. If $2i$ > n, then i has no left child.
- *rightChild*(i) is at $2i+1$ if $2i+1 \le n$. If $2i+1$ >n, then i has no right child.

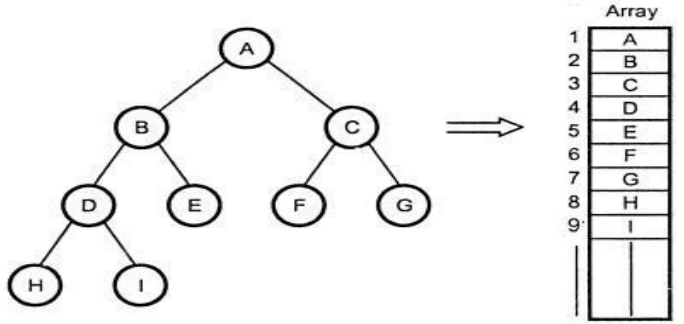


Fig: Array Representation

- **Advantages**
  - Direct Access
  - Finding the Parent / Children is fast
- **Disadvantages**
  - Wastage of memory
  - Insertion and Deletion will be costlier
  - Array size and depth

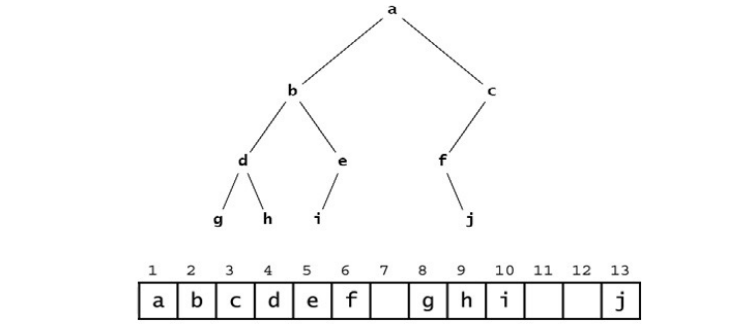


Fig: Array Representation (Disadvantages)

2. Linked List Representation

```
struct Node {
   int info;
   struct Node *LChild;
   struct Node *RChild;
};
typedef struct Node *NODE;
```

Advantages

- No wastage of memory
- Insertion and Deletion will be easy

Disadvantages

- Does not provide direct access
- Additional space in each node.

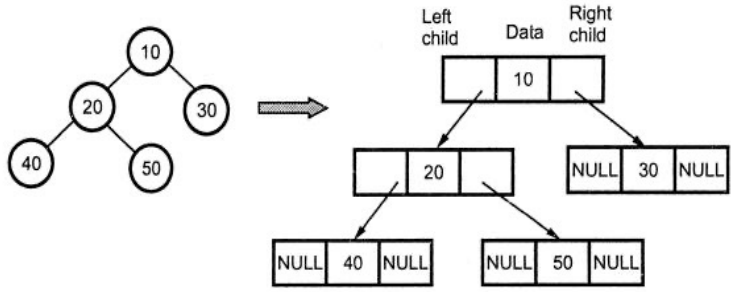
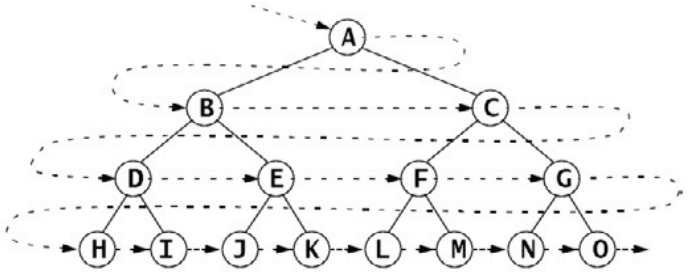


Fig: Linked List Representation

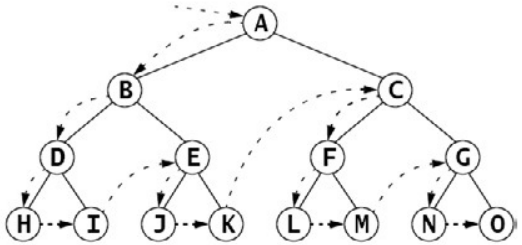
Basic Data Structure (Tree)

Binary Tree Traversal

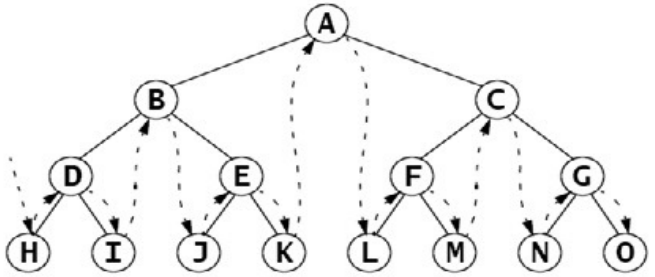
1. Level Order Traversal
2. Pre Order Traversal
3. In Order Traversal
4. Post Order Traversal



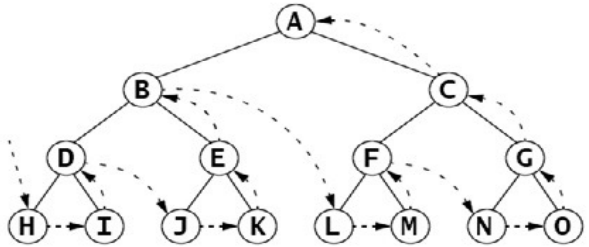
Level Order Traversal: ABCDEFGHIJKLMNO



Pre Order Traversal: ABDHIEJKCFLMGNO



In Order Traversal: HDIBJEKALFMCNGO



Post Order Traversal: HIDJKEBLMENOGCA

Advantages of Tree Data Structure:

1. Efficient for hierarchical data.
2. Fast search, insert, delete operations (e.g., Binary Search Tree).

Disadvantages of Tree DS:

1. Complex implementation.
2. Balancing is needed for performance optimization.

Applications:

1. File systems and databases.
2. Decision-making systems.

Basic Data Structure (Tree (BST))

Binary Search Tree (BST)

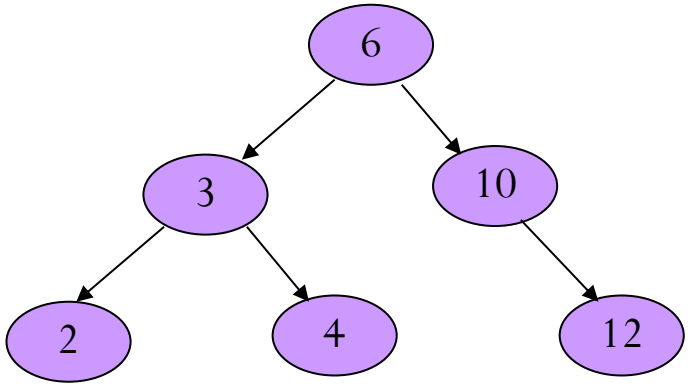
- A Binary Search Tree is a binary tree, which is either

  empty or satisfies the following properties:

  1. Every node has a value and no two nodes have the same value (i.e., all the values are unique).
  2. If there exists a left child or left sub tree then its value is less than the value of the root.
  3. The value(s) in the right child or right sub tree is larger than the value of the root node.

- A Binary Search Tree can be used to store items in its vertices
- It enables efficient searches

Example: Insert the elements 6, 3, 4, 2, 10,12 in that order . The first value to be inserted is put into the root



Insert 6

Insert 3

Insert 4

Insert 2

Insert 10

Insert 12

Code Sample for Insertion of Node in BST (Recursive)

```
struct node *insert(struct node *root, int val) {
        if(root == NULL)
                return getNewNode(val);
        if(root->key < val)
                root->right = insert(root->right,val);
        else if(root->key > val)
                root->left = insert(root->left,val);
        return root;
}
```

Basic Data Structure (Tree (BST))

Insertion of Node in BST (Iterative)

Steps

1. Start from the root and run a loop until a null pointer is reached.
2. Keep the previous pointer of the current node stored.
3. If the current node is null then create and insert the new node there and make it as one of the children of the parent/previous node depending on its value.
4. If the value of current node is less than the new value then move to the right child of current node else move to the left child.

Time Complexity: O(H), where H is the height of the BST.

```
Node* insert(Node* root, int key)
{
    Node* newnode = newNode(key);
    Node* x = root;
    //Pointer y maintains the trailing pointer of x
    Node* y = NULL;
    while (x != NULL) {
        y = x;
        if (key < x->key)
            x = x->left;
        else
            x = x->right;
    }
    //If the root is NULL i.e the tree is empty. The new
node is the root node
    if (y == NULL)
        y = newnode;
    // If the new key is less than the leaf node key.
Assign the new node to be its left child
    else if (key < y->key)
        y->left = newnode;
    // else assign the new node its right child
    else
        y->right = newnode;
    // Returns the pointer where the new node is
inserted
    return y;
}
```

Basic Data Structure (Tree (BST))

Deletion in Binary Search Tree (BST)

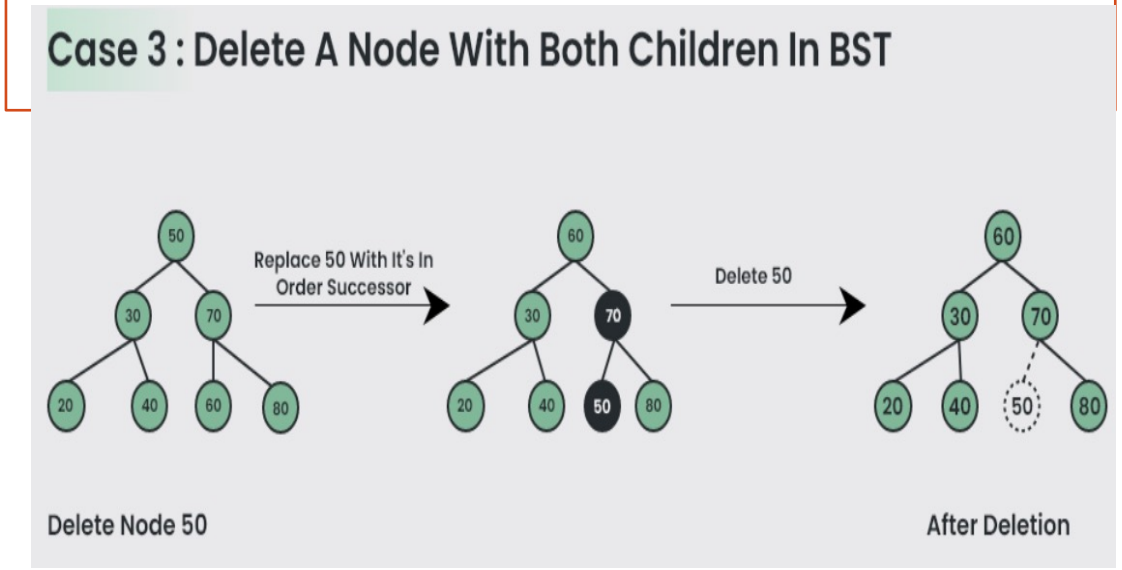
Case 1. Delete a Leaf Node in BST

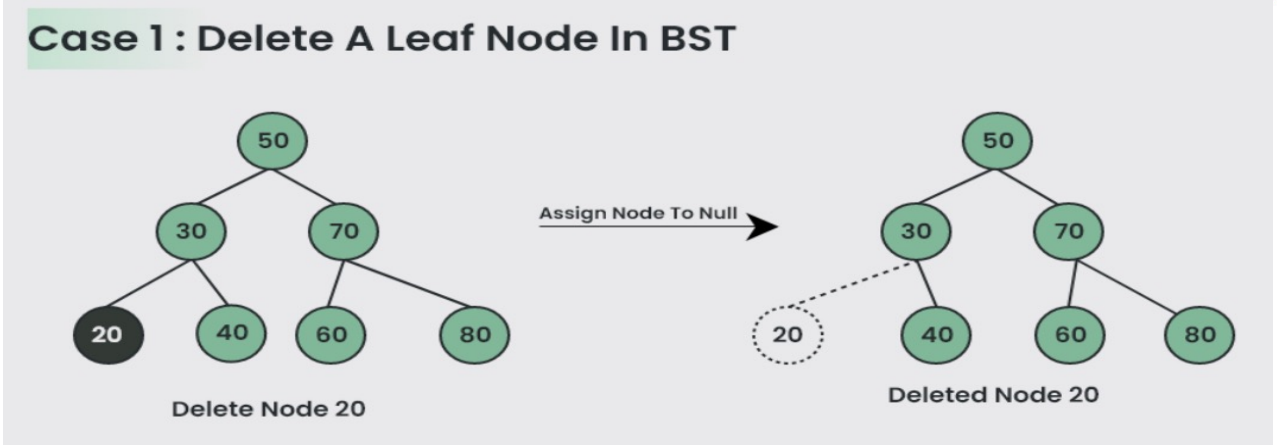
Case 2. Delete a Node with Single Child in BST

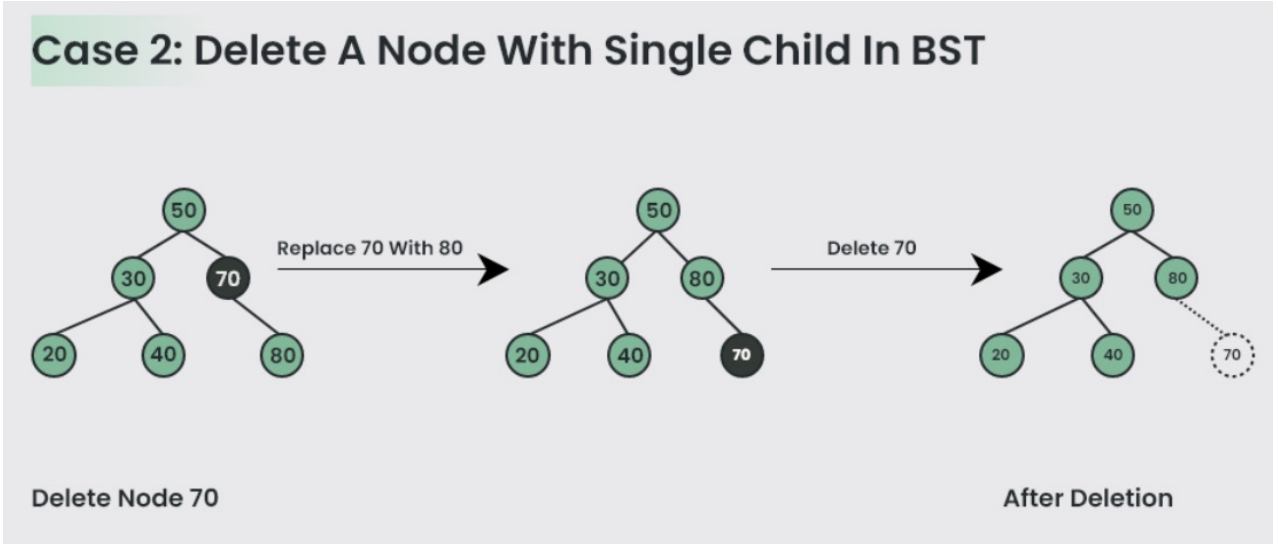
- Copy the child to the node and delete the node.

Case 3. Delete a Node with Both Children in BST

- The trick is to find the inorder successor of the node. Copy contents of the inorder successor to the node, and delete the inorder successor.







Basic Data Structure (Tree (BST))

Deletion operation in a BST (Recursive)

```
struct Node* delNode(struct Node* root, int x)
{
    // Base case
    if (root == NULL)
        return root;

    // If key to be searched is in a subtree
    if (root->key > x)
        root->left = delNode(root->left, x);
    else if (root->key < x)
        root->right = delNode(root->right, x);
    else {
        // If root matches with the given key
        // Cases when root has 0 children or
        // only right child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
```

```
//When root has only left child
        if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }

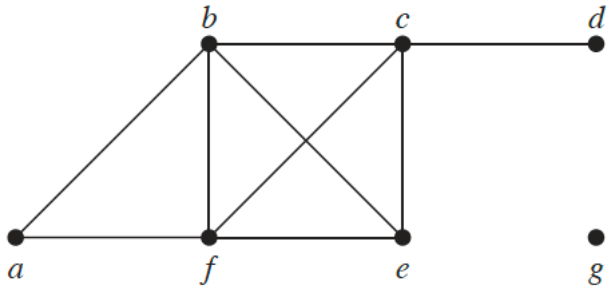
// When both children are present
   struct Node* succ = getSuccessor(root);
   root->key = succ->key;
   root->right = delNode(root->right,succ->key);
    }
    return root;
}
```

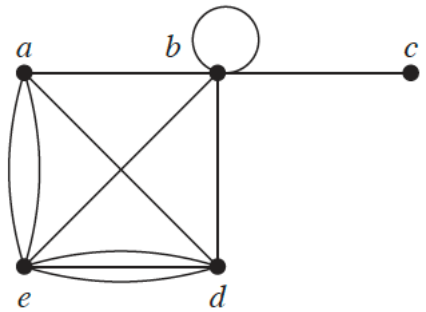
Time Complexity: O(h), where h is the height of the BST.

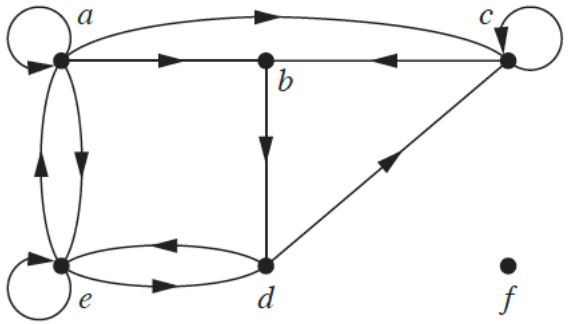
Basic Data Structure (Graph)

Basic Data Structure (Graph)

- Graphs are discrete structures consisting of vertices and edges that connect these vertices.
- There are different kinds of graphs, depending on whether edges have directions, whether multiple edges can connect the same pair of vertices, and whether loops are allowed.
- ***A graph G = (V , E) consists of V, a nonempty set of vertices (or nodes) and E, a set of edges. Each edge has either one or two vertices associated with it, called its endpoints. An edge is said to connect its endpoints.***
- The set of vertices V of a graph G may be infinite. A graph with an infinite vertex set or an infinite number of edges is called an ***infinite graph***, and in comparison, a graph with a finite vertex set and a finite edge set is called a **finite graph**.





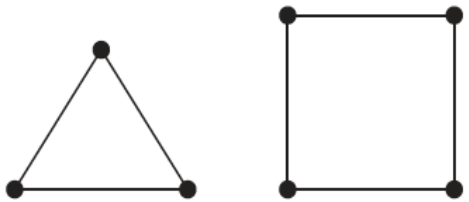


Basic Data Structure (Graph)

Types of graphs:

1. **Undirected Graph:** A Graph whose edges are undirected is called undirected Graph.

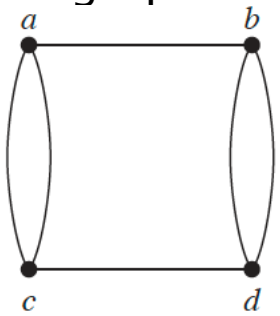
   a. ***Simple Graph:*** A graph in which each edge connects to the two different vertices and no two edges connect same pair of vertices is called a Simple Graph[no- parallel edges and no loops]

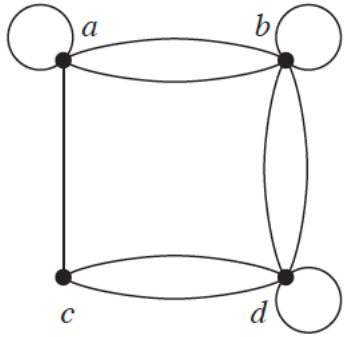


Maximun number of edges possible in a simple graph with n vertices is

$$\frac{n(n-1)}{2}$$

   b. ***Multi Graph:*** If in a graph multiple edges between the same set of vertices are allowed, it is called multigraph.





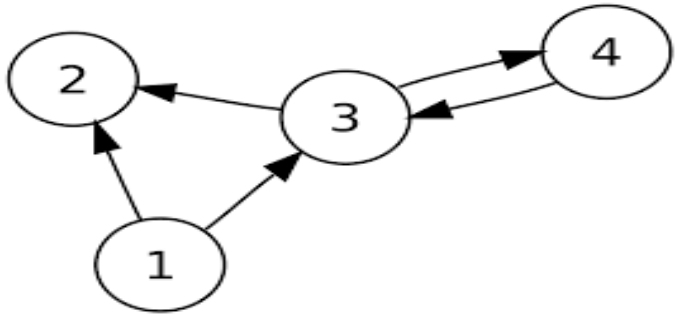
   c. ***Pseudo Graph*** : It is a multigraph with loops.

Basic Data Structure (Graph)

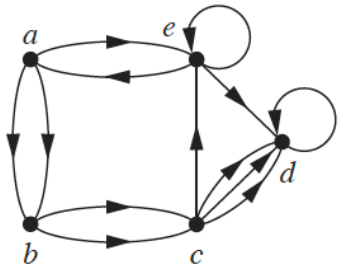
Types of graphs:

1. **Directed Graph:** A Graph whose edges are directed is called directed Graph

   a. ***Simple Directed Graph***: A directed graph in which each edge connects to the two different vertices and no two edges connect same pair of vertices is called a Simple Graph[no- parallel edges and no loops]



   b. ***Multiple Directed Graph***: If in a Directed graph multiple edges between the same set of vertices and loops are allowed, it is called multigraph.



2. **Mixed Graph:** A graph with both directed and undirected edges is called a mixed graph.

Basic Data Structure (Graph)

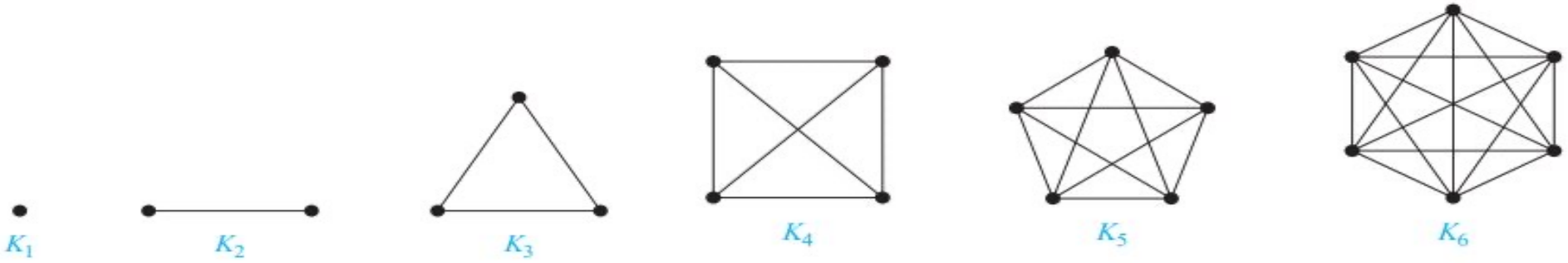
Graph terminologies:

- Two vertices *u* and *v* in an undirected graph *G* are called **adjacent** (or neighbors) in *G* if *u* and *v* are endpoints of an edge *e* of *G*. Such an edge *e* is called incident with the vertices *u* and *v* and *e* is said to connect *u* and *v*.
- The **degree** of a vertex in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex. The degree of the vertex *v* is denoted by deg(*v*).
- A vertex of degree zero is called **isolated**. It follows that an isolated vertex is not adjacent to any vertex.
- A vertex is **pendant** if and only if it has degree one. Consequently, a pendant vertex is adjacent to exactly one other vertex.
- Let (*u*, *v*) be an edge representing edge of a directed graph *G*. *u* is called adjacent to *v* and *v* is called adjacent from *u*. The vertex *u* is called initial vertex and the vertex *v* is called terminal or end vertex. Loop has same initial and terminal vertex.
- In directed graph the in-degree of a vertex *v*, denoted by $\deg^-(v)$, is the number of edges that have *v* as their terminal vertex(incoming edges). The out-degree of a vertex *v*, denoted by $\deg^+(v)$, is the number of edges that have *v* as their initial vertex(outgoing edges). (Note that a loop at a vertex contributes 1 to both the in-degree and the out-degree of this vertex.)

Basic Data Structure (Graph)

Some special types of graphs:

1. **Complete Graph:** A complete graph on *n* vertices, denoted by K_n, is a simple graph that contains exactly one edge between each pair of distinct vertices. The graphs K_n, for n = 1, 2, 3, 4, 5, 6, are displayed in below Figures.



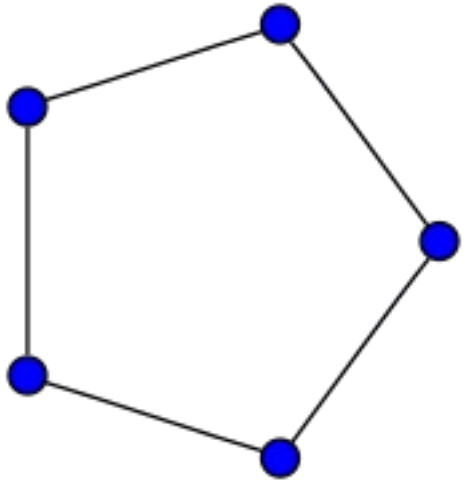
- A complete graph is a regular graph(*a graph where each vertex has the same number of neighbors*) but every regular graph is not complete graph
- A complete graph is a simple graph with max number of edges
- Number of edges in $K_n = \frac{n(n-1)}{2}$
- Degree of each vertex = (n-1)

2. **Cycle Graph**: A **cycle graph** or **circular graph** is a graph that consists of a single cycle, or in other words, some number of vertices (at least 3, if the graph is simple) connected in a closed chain.

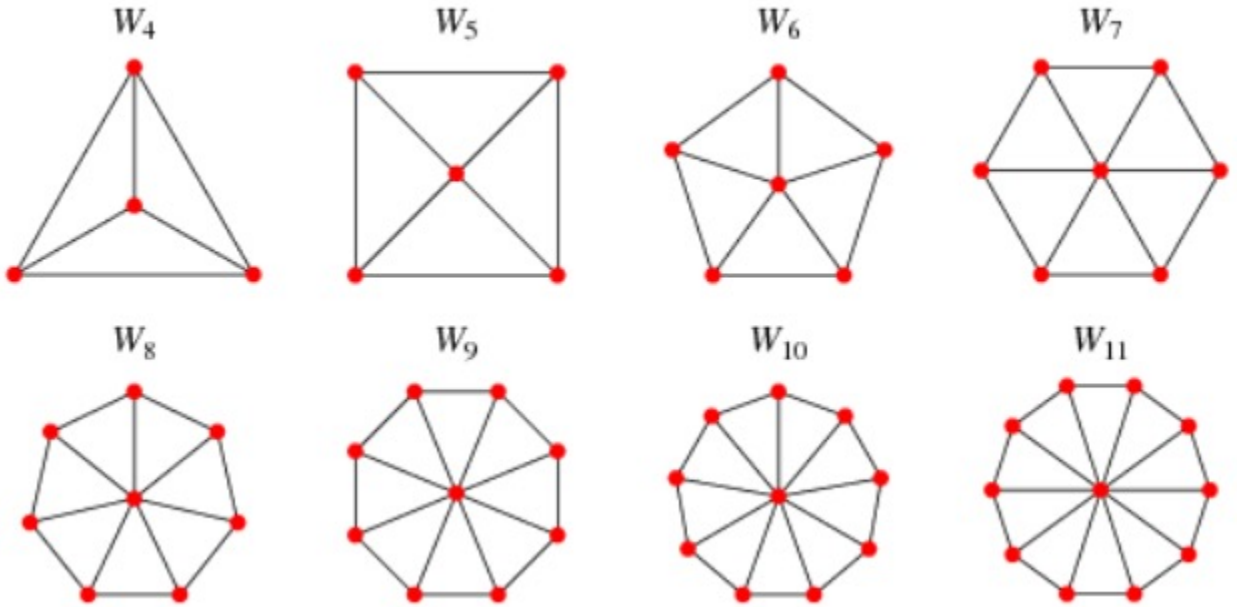
3. **Wheel Graph:** A wheel graph W_n of n vertices(n>=4) can be formed from a cycle graph C_{n-1} by adding a new vertex(hub) which is adjacent to all vertices of C_{n-1}. . The wheel graphs are displayed below.

Basic Data Structure (Graph)

Some special types of graphs:



The cycle graph C_5

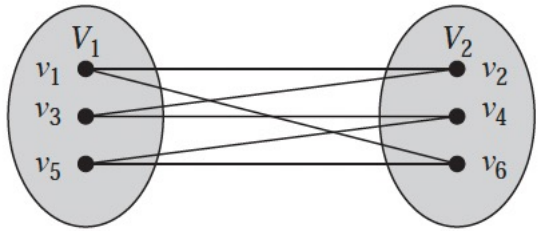


Wheel Graph: Number of edges in $W_n = 2(n-1)$

Basic Data Structure (Graph)

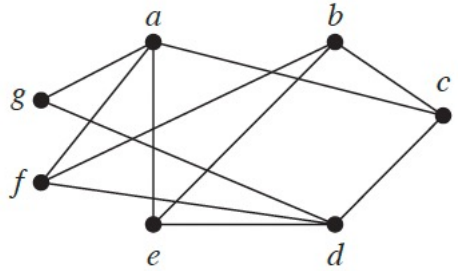
Some special types of graphs:

4. Bipartite Graphs: A simple graph G is called bipartite if its vertex set V can be partitioned into two disjoint sets V_1 and V_2 such that every edge in the graph connects a vertex in V_1 and a vertex in V_2 (so that no edge in G connects either two vertices in V_1 or two vertices in V_2). When this condition holds, we call the pair (V_1, V_2) a bipartition of the vertex set V of G.



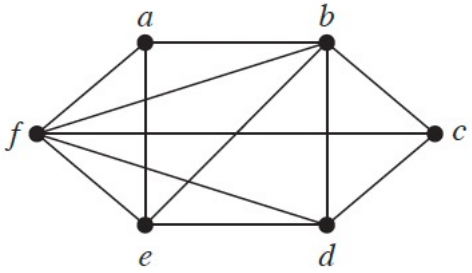
C_6 is Bipartite.

Is this graph is bipartite?



***Yes**, because its vertex set is the union of two disjoint sets, {a, b, d} and {c, e, f, g}, and each edge connects a vertex in one of these subsets to a vertex in the other subset.[**It is not necessary that every vertex in {a, b, d} be adjacent to every vertex in {c, e, f, g}. b and g are not adjacent.**]*

Is this graph is bipartite?



***No**, because its vertex set cannot be partitioned into two subsets so that edges do not connect two vertices from the same subset.*

Basic Data Structure (Graph)

Some special types of graphs:

5. Complete Bipartite Graphs: A complete bipartite $K_{m,n}$ graph is a special type of bipartite graph where every vertex of one set is connected to every vertex of another set.



Fig: Some Complete Bipartite Graphs.

Application: *Bipartite graphs can be used to model many types of applications that involve matching the elements of one set to elements of another.*

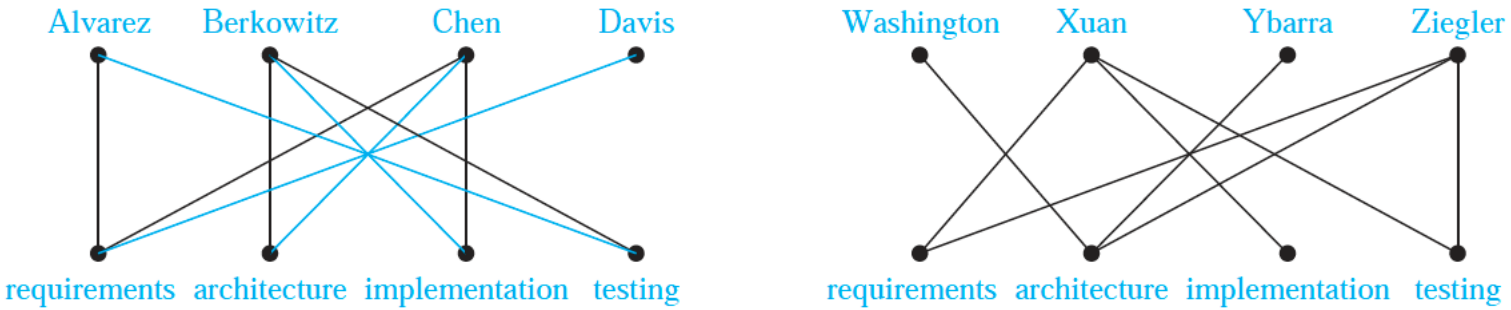
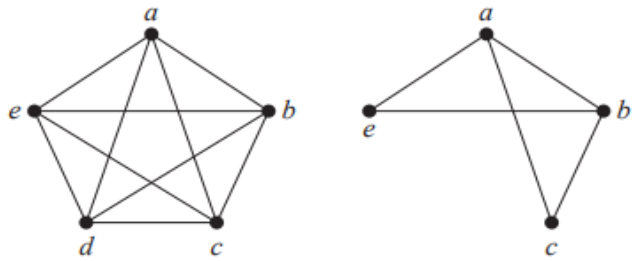


Fig: Modeling the Jobs for Which Employees Have Been Trained

Basic Data Structure (Graph)

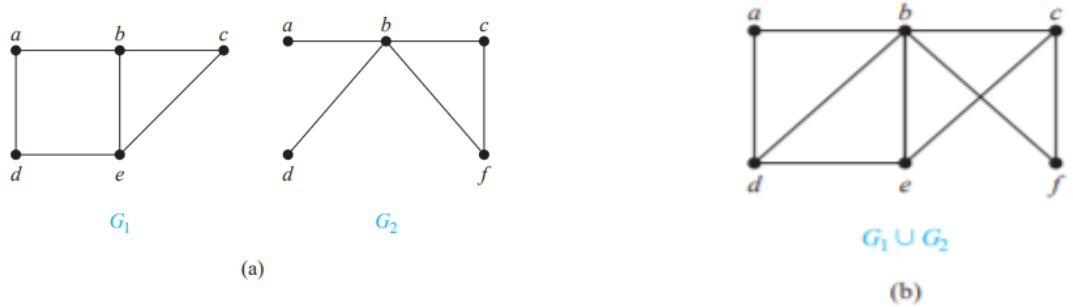
Some special types of graphs:

5. Subgraph Graphs: A subgraph of a graph G = (V , E) is a graph H = (W, F), where W ⊆ V and F ⊆ E. A subgraph H of G is a proper subgraph of G if H != G



A Subgraph of K_5.

GRAPH UNIONS: Two or more graphs can be combined in various ways. The new graph that contains all the vertices and edges of these graphs is called the union of the graphs. The union of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of G_1 and G_2 is denoted by $G_1 \cup G_2$.

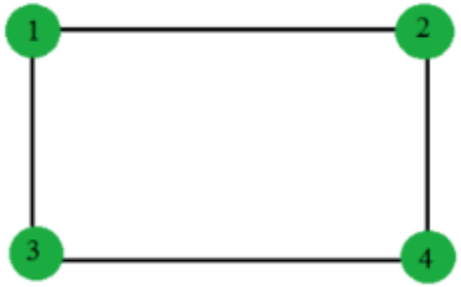


(a) The Simple Graphs G_1 and G_2;

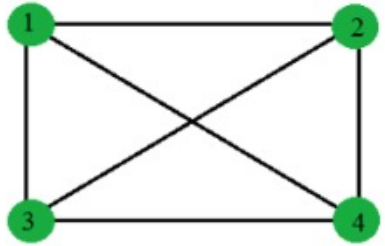
Basic Data Structure (Graph)

Some special types of graphs:

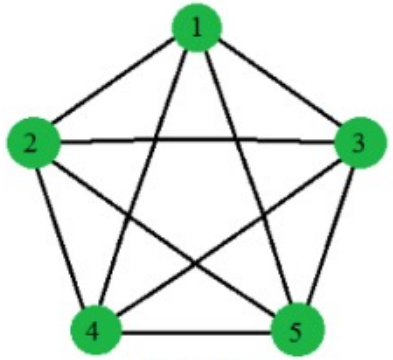
6. **Regular Graphs:** A graph is called regular graph if degree of each vertex is equal. A graph is called **K regular** if degree of each vertex in the graph is K.
   - A complete graph with N vertices is *(N-1)* regular.
   - Cycle(C_n) is always 2 Regular.



2 Regular



3 Regular



4 Regular

Basic Data Structure (Graph)

Representing Graphs:

1. Adjacency List:

- One way to represent a graph is to use adjacency lists, which specify the vertices that are adjacent to each vertex of the graph

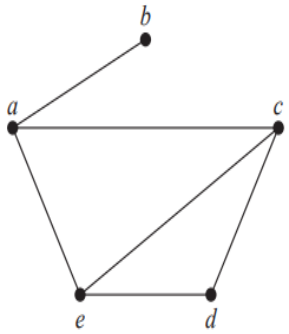


FIGURE 1 A Simple Graph.

TABLE 1 An Adjacency List for a Simple Graph.

Vertex	Adjacent Vertices
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d

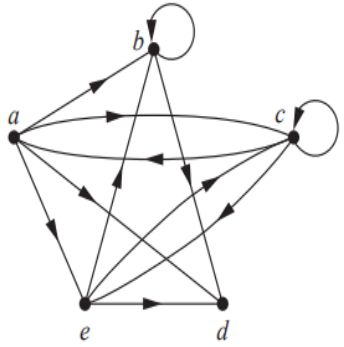


FIGURE 2 A Directed Graph.

TABLE 2 An Adjacency List for a Directed Graph.

Initial Vertex	Terminal Vertices
a	b, c, d, e
b	b, d
c	a, c, e
d	
e	b, c, d

```
struct Node {
 int vertex;
 struct Node* next;
};
struct Graph {
 int numVertices;
 struct Node** adjLists;
};
```

```
struct Node* createNode(int v)
{
  struct Node* newNode= malloc
  (sizeof(struct Node));
  newNode->vertex = v;
  newNode->next = NULL;
  return newNode;
}
```

```
struct Graph* createGraph(int vertices){
 struct Graph* graph = malloc(sizeof(structGraph));
 graph->numVertices=vertices;
 graph->adjLists =
 malloc(vertices*sizeof(struct Node*));
for (int i = 0; i < vertices; i++)
  graph->adjLists[i] = NULL;
return graph;
}
```

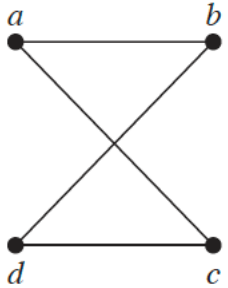
Basic Data Structure (Graph)

Representing Graphs:

2. Adjacency Matrix:

- Suppose that G = (V , E) is a simple graph where |V| = n. Suppose that the vertices of G are listed arbitrarily as v_1, v_2,..., v_n. The **adjacency matrix** A (or AG) of G, with respect to this listing of the vertices, is the n x n zero–one matrix with 1 as its (i, j)th entry when v_i and v_j are adjacent, and 0 as its (i, j)th entry when they are not adjacent. In other words, if its adjacency matrix is A = [a_{ij}], then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

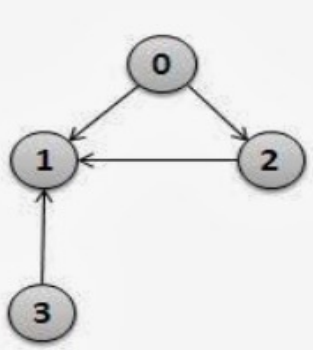
with respect to the ordering of vertices a, b, c, d.

Sample Code

```
#define V 4
int mat[V][V] = {
        {0, 1, 0, 0},
        {1, 0, 1, 0},
        {0, 1, 0, 1},
        {0, 0, 1, 0}
    };
```

Basic Data Structure (Graph)

Representing Graphs:



	0	1	2	3
0	0	1	1	0
1	0	0	0	0
2	0	1	0	0
3	0	1	0	0

Adjacency Matrix Representation of Directed Graph

The matrix for a directed graph G = (V , E) has a 1 in its (i, j)th position if there is an edge from v_i to v_j , where v_1, v_2,..., v_n is an arbitrary listing of the vertices of the directed graph. In other words, if A = [a_{ij}] is the adjacency matrix for the directed graph with respect to this listing of the vertices, then

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from v_j to v_i when there is an edge from v_i to v_j .

Basic Data Structure (Graph)

Representing Graphs:

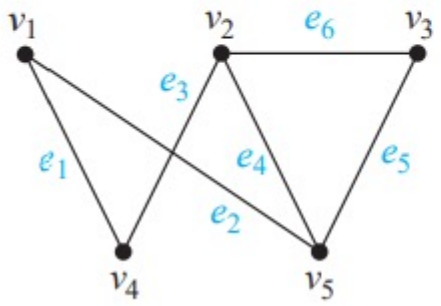
3. Incidence Matrices:

Another common way to represent graphs is to use **incidence matrices**. Let $G = (V, E)$ be an undirected graph. Suppose that $v_1, v_2, \ldots, v_n$ are the vertices and $e_1, e_2, \ldots, e_m$ are the edges of G. Then the incidence matrix with respect to this ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

Represent the graph shown in Figure 6 with an incidence matrix.

Solution: The incidence matrix is



$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{array}.$$

Basic Data Structure (Graph)

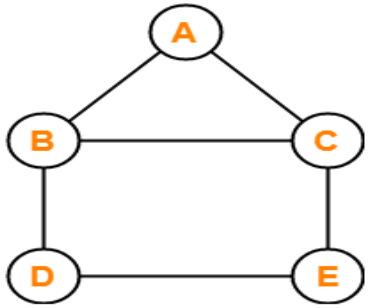
Connectivity:

- **Connectivity** is a basic concept of graph theory. It defines whether a graph is connected or disconnected. Without connectivity, it is not possible to traverse a graph from one vertex to another vertex.

1. **WALK:** A walk is defined as a finite alternating sequence of vertices and edges, beginning and ending with vertices such that each edge is incident with the vertices preceding and following it.
   - **Open Walk:** a walk is called as an Open walk if Length of the walk is greater than zero and the vertices at which the walk starts and ends are different.
   - **Closed Walk:** a walk is called as an Closed walk if Length of the walk is greater than zero and the vertices at which the walk starts and ends are same.
- If length of walk i= 0, then it is called as a Trivial Walk
- Both vertices and edges can repeat in a walk whether it is an open walk or a closed walk.

In this graph, few examples of walk are-

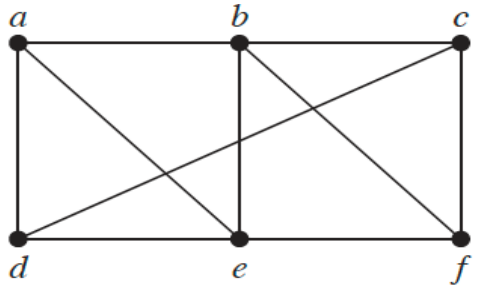
- a , b , c , e , d (Length = 4)
- d , b , a , c , e , d , e , c (Length = 7)
- e , c , b , a , c , e , d (Length = 6)



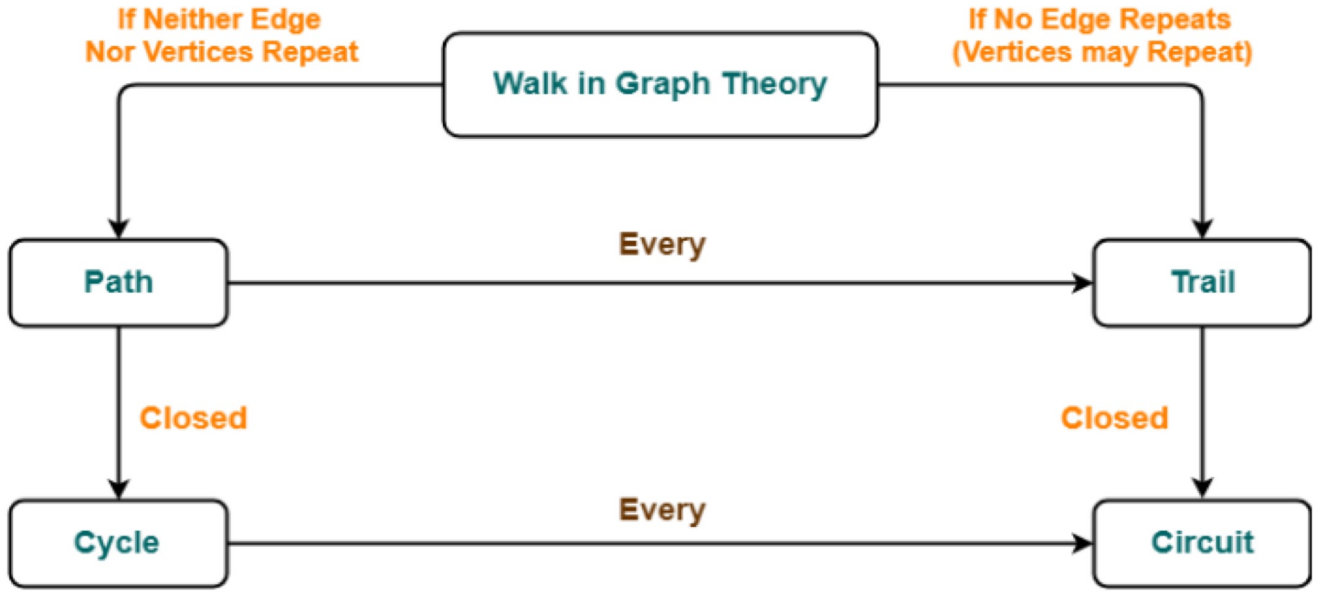
Basic Data Structure (Graph)

Connectivity:

2. **TRIAL:** A trail is defined as an open walk in which Vertices may repeat but edges are not allowed to repeat.
3. **CIRCUIT:** A circuit is defined as a closed walk in which Vertices may repeat but edges are not allowed to repeat.(Closed trial)
4. **PATH:** A path is defined as an open walk in which neither vertices are allowed to repeat nor edges are allowed to repeat.
5. **CYCLE:** A cycle is defined as a closed walk in which neither vertices (except possibly the starting and ending vertices) are allowed to repeat nor edges are allowed to repeat.(Closed Path)



Here, a, d, c, f , e is a path of length 4
b, c, f , e, b is a circuit of length 4 (start and end at b) The path a, b, e, d, a, b, which is of length 5, is not simple because it contains the edge {a, b} twice.



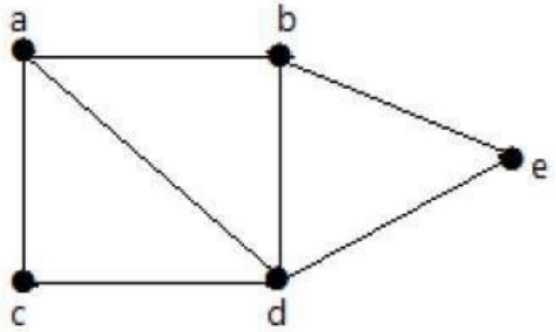
Important Chart to Remember

Basic Data Structure (Graph)

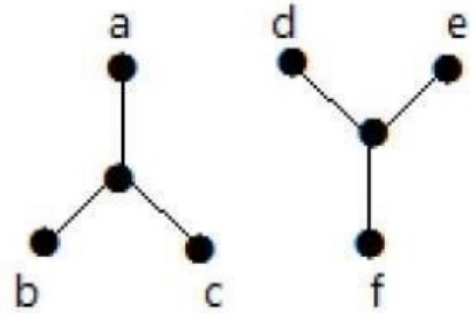
Connectedness in Undirected Graphs:

- A graph is said to be **connected if there is a path between every pair of vertex**. From every vertex to any other vertex, there should be some path to traverse. That is called the connectivity of a graph

Example 1: In the following graph, it is possible to travel from one vertex to any other vertex. For example, one can traverse from vertex 'a' to vertex 'e' using the path 'a-b-e'.



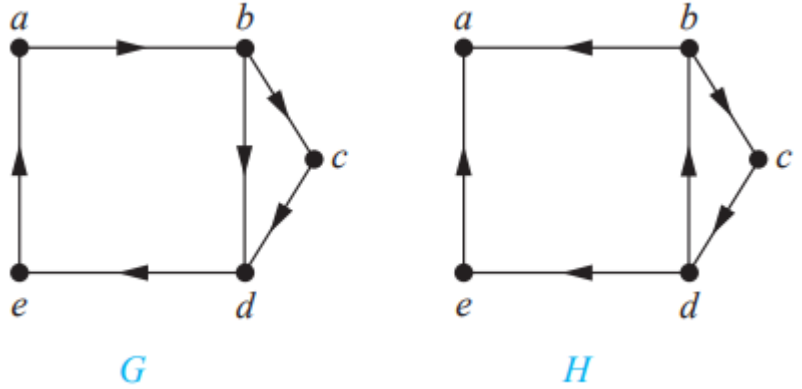
Example 2 : In the following example, traversing from vertex 'a' to vertex 'f' is not possible because there is no path between them directly or indirectly. Hence it is a disconnected graph.



Basic Data Structure (Graph)

Connectedness in directed Graphs:

- A directed graph is **strongly connected** if there is a path from **a** to **b** and from **b** to **a** whenever **a** and **b** are vertices in the graph.
- A directed graph is **weakly connected** if there is a path between every two vertices in the underlying undirected graph



- Graph G is strongly connected because there is a path between any two vertices in this directed graph.

- The graph H is not strongly connected. There is no directed path from **a** to **b** in this graph. However, H is weakly connected, because there is a path between any two vertices in the underlying undirected graph of H

Basic Data Structure (Graph)

Applications of Graph Connectivity

1. **Network Design and Analysis**
   - **Communication Networks**: Ensuring that data can travel between any two nodes in a network. Applications include the design of internet, telecommunication, and social networks.
   - **Transportation Networks**: Ensuring connectivity in road, railway, or airline networks to guarantee access to all locations.
   - **Power Grids**: Analyzing the robustness of power grids to failures or attacks, ensuring continuous electricity supply.
2. **Social Network Analysis**
   - Determining clusters or communities within a social network.
   - Identifying influential individuals based on how well-connected they are to others.
   - Understanding the spread of information, rumors, or influence in a social graph.
3. **Infrastructure Planning**
   - Urban planning: Designing road systems or pipelines where connectivity ensures accessibility and efficiency.
   - Disaster recovery: Evaluating the vulnerability of infrastructure to disconnection in case of natural disasters.
4. **Biological Networks**
   - Analyzing protein interaction networks to understand biological processes.
   - Studying neural networks to understand connectivity patterns in the brain.
   - Modeling and studying ecosystems through food web analysis.
5. **Data Structures and Algorithms**
   - Designing efficient algorithms for problems like shortest path, spanning trees, or clustering.
   - Applications in computer graphics and computational geometry (e.g., ensuring polygon connectivity).

Basic Data Structure (Graph)

Applications of Graph Connectivity

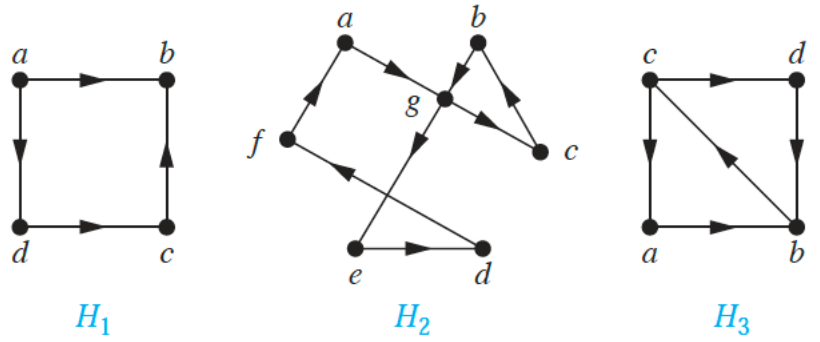
6. **Web Crawling and Search Engines**
   - Ensuring a crawler can reach all parts of the web graph.
   - Studying the structure of the web to improve search and ranking algorithms.
7. **Security and Robustness**
   - **Cybersecurity**: Identifying weak points in networks that might lead to partitioning when nodes are attacked.
   - **Fault Tolerance**: Designing systems that maintain connectivity even when some components fail.
8. **Operations Research**
   - Optimizing logistics and supply chain networks for connected and efficient delivery systems.
   - Modeling and solving problems like the traveling salesman problem or vehicle routing problem.
9. **Artificial Intelligence and Machine Learning**
   - Graph-based semi-supervised learning, where connectivity is used to propagate labels.
   - Knowledge graph analysis, where relationships between entities depend on graph connectivity.
10. **Telecommunication and Sensor Networks**
    - Ensuring reliability and reach in cellular networks, sensor networks, and IoT systems.
    - Designing relay and routing strategies based on graph connectivity.
    - Each of these applications leverages the concept of connectivity in different ways to solve domain-specific problems or optimize existing systems.

Basic Data Structure (Graph)

EULER GRAPH:

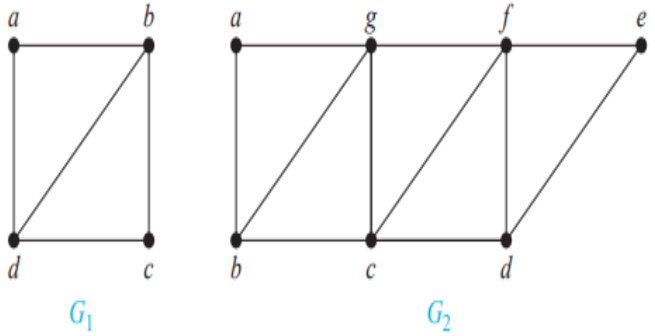
1. An **Euler path** is a path in a graph that visits every edge **exactly once**. The path may visit vertices multiple times, but each edge is traversed only once. If such a path exists and starts and ends at different vertices, it is called an **Euler path**. If it starts and ends at the same vertex, it becomes an **Euler circuit** or **Eulerian cycle**.
2. A graph containing Euler Path is called **Euler Graph**.

Eg: The graph H2 has an Euler circuit, for example, a, g, c, b, g, e, d, f, a. Neither H1 nor H3 has an Euler circuit. H3 has an Euler path, namely, c, a, b, c, d, b, but H1 does not.



Necessary and Sufficient Conditions for Euler Circuits And Paths:

1. *A connected multigraph with at least two vertices has an Euler circuit if and only if each of its vertices has even degree.*
2. *A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree and they are starting and ending point.*



G1 contains exactly two vertices of odd degree, namely, **b** and **d**. Hence, it has an Euler path that must have **b** and **d** as its endpoints. One such Euler path is **d, a, b, c, d, b.** Similarly, G2 has exactly two vertices of odd degree, namely, b and d. So it has an Euler path that must have b and d as endpoints. One such Euler path is **b, a, g, f, e, d, c, g, b, c, f, d.** [*Illustrate Euler circuit (point 1) using H_2. In H_2, Every vertex contains even degree, every vertex incoming and outgoing degree are equal, it is multigraph and strongly connected as well. So one of the Euler circuit is a,g,c,b,g,e,d,f,a]*

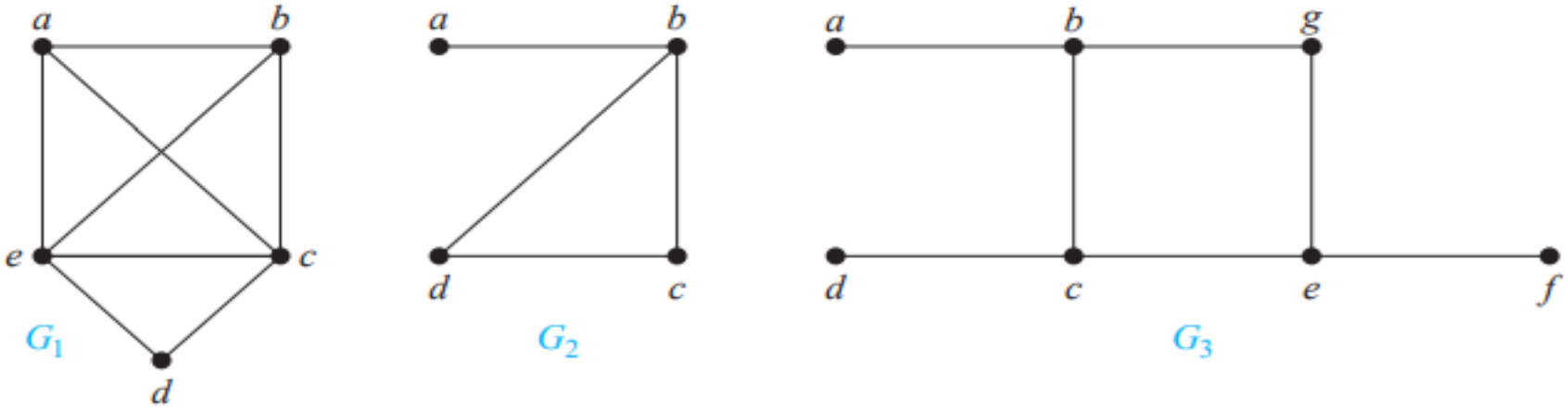
Basic Data Structure (Graph)

Hamilton Graphs:

- A path in a graph G that passes through every vertex exactly once is called a **Hamilton path**. The Hamilton Circuit is a special type of Hamilton path. When the starting vertex of the Hamilton path is also connected with the ending vertex of that path, then it is called the **Hamilton Circuit**.

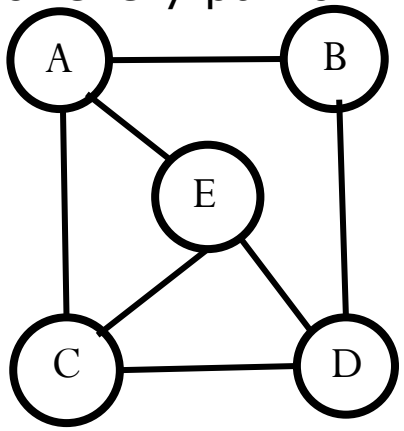
Which of the simple graphs in Figure 10 have a Hamilton circuit or, if not, a Hamilton path?

Solution: G_1 has a Hamilton circuit: a, b, c, d, e, a. There is no Hamilton circuit in G_2 (this can be seen by noting that any circuit containing every vertex must contain the edge $\{a, b\}$ twice), but G_2 does have a Hamilton path, namely, a, b, c, d. G_3 has neither a Hamilton circuit nor a Hamilton path, because any path containing all vertices must contain one of the edges $\{a, b\}$, $\{e, f\}$, and $\{c, d\}$ more than once.



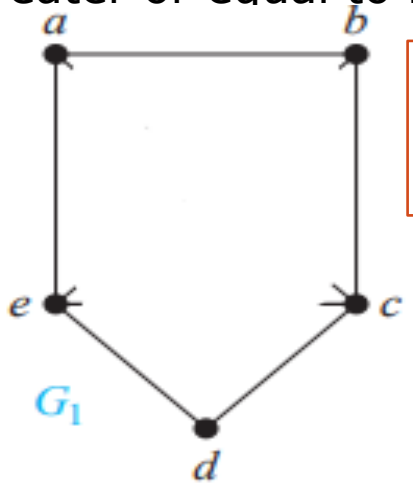
Sufficient Condition for Hamilton Graph:

1. **ORE'S THEOREM :** If G is a simple graph with n vertices with **n ≥ 3** such that the sum of degrees of every pair of non adjacent vertices is greater or equal to n, then G has Hamilton circuit.



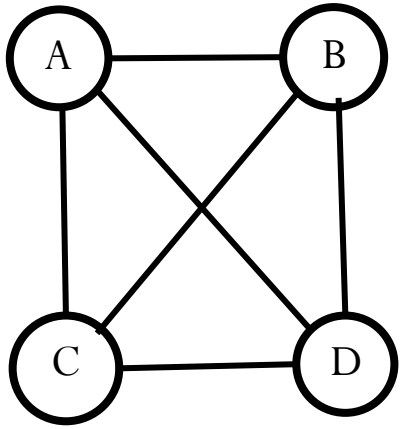
Total vertices(n) = 5
- AD = 3+3 = 6 ≥ 5
- BC = 2+3 = 5 ≥ 5
- BE = 2+3 = 5 ≥ 5

Hence it is Hamiltonian.

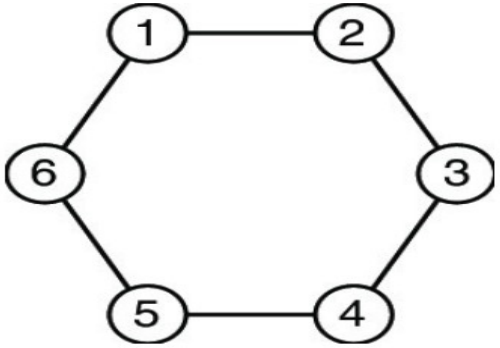


ORE's Condition does not apply to this graph but this graph is still Hamiltonian.

2. **DIRAC'S THEOREM :** If G is a simple graph with n vertices with **n ≥ 3** such that the degree of every vertex in G is at least n/2, then G has a Hamilton circuit.



Total vertices(n) = 4
- Degree of A = 3≥4/2=2
- Degree of B = 3≥4/2=2
- Degree of C = 3≥4/2=2
- Degree of D = 3≥4/2=2
- Hence it is Hamiltonian.



DIRAC's Condition does not apply to this graph but this graph is still Hamiltonian.

Algorithm Design Approaches

1. Divide and Conquer
2. Greedy Algorithm
3. Backtracking Algorithms
4. Branch & Bound
5. Randomized Algorithms

Algorithm

- An algorithm is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer.
- The formal definition of an algorithm is that it contains the finite set of instructions which are being carried in a specific order to perform the specific task.
- It is not the complete program or code; it is just a solution (logic) of a problem, which can be represented either as an informal description using a Flowchart or Pseudocode.

Characteristics of an Algorithm

- **Input:** An algorithm has some input values. We can pass some input value to an algorithm.
- **Output:** We will get 1 or more output at the end of an algorithm.
- **Unambiguity:** An algorithm should be unambiguous which means that the instructions in an algorithm should be clear and simple.

Algorithm

- **Finiteness:** An algorithm should have finiteness. Here, finiteness means that the algorithm should contain a limited number of instructions, i.e., the instructions should be countable.
- **Effectiveness:** An algorithm should be effective as each instruction in an algorithm affects the overall process.
- **Language independent:** An algorithm must be language-independent so that the instructions in an algorithm can be implemented in any of the languages with the same output.

Why Algorithm?

- **Scalability:** It helps us to understand the scalability. When we have a big real-world problem, we need to scale it down into small-small steps to easily analyze the problem.
- **Performance:** The real-world is not easily broken down into smaller steps. If the problem can be easily broken into smaller steps means that the problem is feasible.

Algorithm

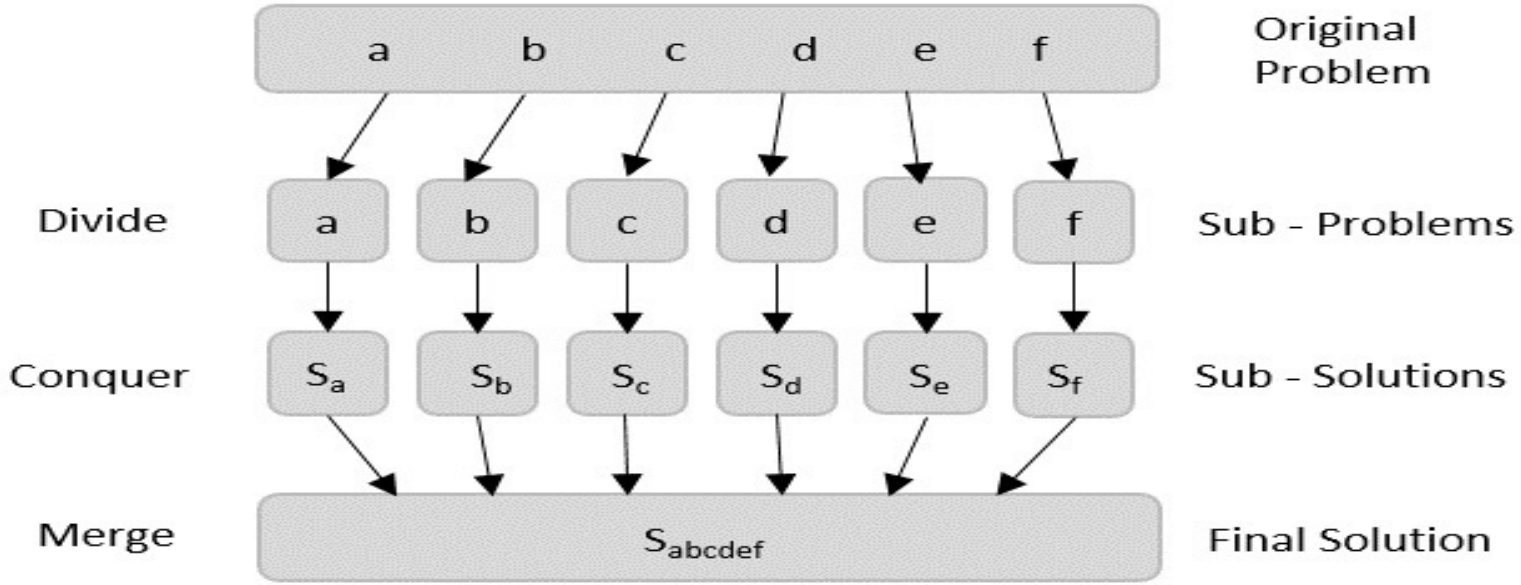
Factors of an Algorithm

- **Modularity:** If any problem is given and we can break that problem into small modules or small steps, which is a basic definition of an algorithm, it means that this feature has been perfectly designed for the algorithm.
- **Correctness:** The correctness of an algorithm is defined as when the given inputs produce the desired output, which means that the algorithm has been designed algorithm. The analysis of an algorithm has been done correctly.
- **Maintainability:** Maintainability means that the algorithm should be designed in a very simple structured way so that when we redefine the algorithm, no major change will be done in the algorithm.
- **Functionality:** It considers various logical steps to solve the real-world problem.
- **Robustness:** Robustness means that how an algorithm can clearly define our problem.
- **User-friendly:** If the algorithm is not user-friendly, then the designer will not be able to explain it to the programmer.
- **Simplicity:** If the algorithm is simple then it is easy to understand.
- **Extensibility:** If any other algorithm designer or programmer wants to use your algorithm then it should be extensible.

Approaches of Algorithm

1. Divide and Conquer

- Divide the problem into smaller subproblems, solve independently, and combine results.
- A divide and conquer algorithm is a method to solve big problems by breaking them down into smaller, more manageable parts.
- The idea is to "divide" the problem into smaller pieces, "conquer" each piece by solving it, and then ”merge" the solutions to get the final answer.
- Example : **Merge Sort**, **Quick Sort**, **Binary Search**



Approaches of Algorithm

2. Greedy Algorithm

- It is an algorithm paradigm that makes an optimal choice on each iteration with the hope of getting the best solution.
- It is easy to implement and has a faster execution time. But, there are very rare cases in which it provides the optimal solution.
- Example : **Dijkstra's Shortest Path**, **Kruskal's MST**

3. Backtracking

- Build a solution incrementally, abandoning unfeasible solutions.
- It is an algorithmic technique that solves the problem recursively and removes the solution if it does not satisfy the constraints of a problem.
- It involves finding a solution incrementally by trying different options and undoing them if they lead to a dead end.
- It is commonly used in situations where you need to explore multiple possibilities to solve a problem, like searching for a path in a maze or solving puzzles like Sudoku. When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted.
- Example : **Sudoku, crossword puzzles, N-Queens**

Approaches of Algorithm

4.Branch and Bound Algorithm

- This approach divides all the sets of feasible solutions into smaller subsets. These subsets are further evaluated to find the best solution.
- It is a method used in **combinatorial** optimization problems to systematically search for the best solution.
- The branch and bound approach is based on the principle that the total set of feasible solution can be partitioned into smaller subsets of solutions. These smaller subsets can then be evaluated systematically until the best solution is found.
- Example : **Traveling Salesman Problem, Job scheduling**.

Approaches of Algorithm

5.Randomized Algorithm

- A **randomized algorithm** is a technique that uses a source of randomness as part of its logic.
- It is typically used to reduce either the running time, or time complexity; or the memory used, or space complexity, in a standard algorithm.
- The algorithm works by generating a random number, ***r***, within a specified range of numbers, and making decisions based on ***r***'s value.
- Unlike deterministic algorithms, randomized algorithms consider randomized bits of the logic along with the input that in turn contribute towards obtaining the output.
- Due to the potential erroneous output of the algorithm, an algorithm known as **amplification** is used in order to boost the probability of correctness by sacrificing runtime.
- Amplification works by repeating the randomized algorithm several times with different random subsamples of the input, and comparing their results.
- It is common for randomized algorithms to amplify just parts of the process, as too much amplification may increase the running time beyond the given constraints.

Approaches of Algorithm

5.Randomized Algorithm

Two Forms of Randomized algorithms

1. **Las Vegas Algorithm:** A **Las Vegas** algorithm runs within a specified amount of time. If it finds a solution within that time frame, the solution will be exactly correct; however, it is possible that it runs out of time and does not find any solutions.
2. **Monte Carlo Algorithm:** A **Monte Carlo** algorithm is a probabilistic algorithm which, depending on the input, has a slight probability of producing an incorrect result or failing to produce a result altogether.

- In Practical, computers cannot generate completely random numbers, so randomized algorithms in computer science are approximated using a pseudorandom number generator in place of a true source of random number, such as the drawing of a card.

Example: **QuickSort** (random pivot), **Monte Carlo simulations**.

Approaches of Algorithm

6. Dynamic programming

- It makes the algorithm more efficient by storing the intermediate results. It follows five different steps to find the optimal solution for the problem:
  1. It breaks down the problem into a subproblem to find the optimal solution.
  2. After breaking down the problem, it finds the optimal solution out of these subproblems.
  3. Stores the result of the subproblems is known as **memorization**.
  4. Reuse the result so that it cannot be recomputed for the same subproblems.
  5. Finally, it computes the result of the complex program.

Example: **Bellman–Ford Shortest Path**, **Floyd Warshall**, **Edit Distance and Matrix Chain Multiplication.**

Approaches of Algorithm

7. Brute force algorithm

- The general logic structure is applied to design an algorithm.
- It is also known as an exhaustive search algorithm that searches all the possibilities to provide the required solution. Such algorithms are of two types:

1. **Optimizing:** Finding all the solutions of a problem and then take out the best solution or if the value of the best solution is known then it will terminate.
   - Eg: Finding the best path for the travelling salesman problem. Here best path means that travelling all the cities and the cost of travelling should be minimum.

2. **Sacrificing:** It stops finding the solution as soon as the satisfactory solution is found.
   - Eg: Finding the travelling salesman path which is within 10% of optimal.

Analysis of Algorithm

1. Time Complexity
2. Space Complexity
3. Asymptotic notations
   - Big O
   - Big Theta
   - Big Omega

Analysis of Algorithm

Two Level of Algorithm Analysis

1. Priori Algorithm Analysis:

- **Priori analysis** is the theoretical analysis of an algorithm which is done before implementing the algorithm.
- Various factors can be considered before implementing the algorithm like processor speed, which has no effect on the implementation part.
  - Example of A **Priori analysis** is time complexity analysis.
    - Bubble-sort has a time complexity of $O(n^2)$. This is always the same, no matter what the programming language or hardware is.

2. Posterior Algorithm Analysis:

- Posterior analysis is a practical analysis of an algorithm.
- The practical analysis is achieved by implementing the algorithm using any programming language.
- This analysis basically evaluate that how much running time and space taken by the algorithm.

Analysis of Algorithm

Posterior Analysis	Priori Analysis
Posteriori analysis is a relative analysis.	Priori analysis is an absolute analysis.
It is dependent on language of compiler and type of hardware.	It is independent of language of compiler and types of hardware.
It will give exact answer.	It will give approximate answer.
It doesn't use asymptotic notations to represent the time complexity of an algorithm.	It uses the asymptotic notations to represent how much time the algorithm will take in order to complete its execution.
The time complexity of an algorithm using a posteriori analysis differ from system to system.	The time complexity of an algorithm using a priori analysis is same for every system.
If the time taken by the program is less, then the credit will go to compiler and hardware.	If the algorithm running faster, credit goes to the programmer.

Analysis of Algorithm

Why Performance Analysis?

- There are multiple algorithms to solve a problem, so we need to select the best one among them (Eg: finding a shortest path among source to destination).
- Performance analysis helps us to select the best algorithm(analyze them and pick the one which is best suitable for requirements) from multiple algorithms to solve a problem.
- Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.
- To compare, we use a set of parameters or set of elements like memory required by that algorithm, the execution speed of that algorithm, easy to understand, easy to implement, etc.,
- An algorithm depends on
  - Whether that algorithm is providing the exact solution for the problem?
  - Whether it is easy to understand?
  - Whether it is easy to implement?
  - How much space (memory) it requires to solve the problem?
  - How much time it takes to solve the problem? Etc.,

Analysis of Algorithm

Performance analysis on the basis of:

1. Space required to complete the task of that algorithm (**Space Complexity**). It includes program space and data space.
2. Time required to complete the task of that algorithm (**Time Complexity**)

Time Complexity:

- The time complexity of an algorithm is denoted by the big O notation. Big O notation is the **asymptotic notation** to represent the time complexity.
- The time complexity is mainly calculated by counting the number of steps to finish the execution.

```
int sum(int n){
    sum=0;
    for i=1 to n
        sum=sum+i;
    return sum;
}
```

Explanation**: The time complexity of the loop statement will be at least **n**, and if the value of **n** increases, then the time complexity also increases. While the complexity of the code, i.e., return sum will be constant as its value is not dependent on the value of **n** and will provide the result in one step only. We generally consider the worst-time complexity as it is the maximum time taken for any given input size. So the time complexity of sample code is: **O(n)

Analysis of Algorithm

Space complexity:

- An algorithm's **space complexity** is the amount of space required to solve a problem and produce an output. Similar to the time complexity, space complexity is also expressed in big O notation.
- **Why Space is required?**
  - To store program instructions
  - To store constant values
  - To store variable values
  - To track the function calls, jumping statements, etc

  Space complexity = Auxiliary space + Input size.
  - **Auxiliary space**: The extra space required by the algorithm, excluding the input size, is known as an auxiliary space.
- *Total amount of computer memory required by an algorithm to complete its execution is called as **space complexity** of that algorithm.*
- When a program is under execution it uses the computer memory for THREE reasons
  1. **Instruction Space:** It is the amount of memory used to store compiled version of instructions.
  2. **Environmental Stack:** It is the amount of memory used to store information of partially executed functions at the time of function call.
  3. **Data Space:** It is the amount of memory used to store all the variables and constants.

Analysis of Algorithm

- **Space complexity:**
  - To perform algorithm analysis on the basis of its Space complexity, consider only Data Space and ignore Instruction Space and Environmental Stack.
  - i,e: calculate only the memory required to store **Variables**, **Constants**, **Structures**, etc.,

```
int square(int a)
{
    return a*a;
}
```

In example, it requires 4 bytes of memory to store variable 'a' and another 4 bytes of memory is used for return value. In Total, it requires 8 bytes of memory to complete its execution. This 4 bytes of memory is fixed for any input value of 'a'. This space complexity is said to be Constant Space Complexity. So the space complexity is ***O(1).***

```
int sum(int A[], int n) {
    int sum = 0, i;
    for (i = 0; i < n; i++)
        sum = sum + A[i];
  return sum;
}
```

- *If we assume the array A[] is being created as part of the function (or within the program flow), then we need to account for the memory required for A[]:*
- *Size of Array A[]: If A[] contains n integers, the space required is proportional to n. For example:*
  - *If int takes 4 bytes, the array would take 4n bytes.*
  - *Thus, the space complexity would be O(n).*
- *Conclusion: If A[] already exists before the function is called, the space complexity is O(1).If A[] is being created for the first time, the space complexity is O(n) because of the memory required to store the array.*

Analysis of Algorithm

Asymptotic notations

- Big O
- Big Theta
- Big Omega

Complexity Analysis

- Complexity analysis is the systematic study of the cost of a computation, measured either in time units or in operations performed, or in the amount of storage space required.
- The goal is to have a meaningful measure that permits comparison of algorithms and/or implementations independent of operating platform.
- Complexity analysis involves two distinct phases:
  - **algorithm analysis**: analysis of the algorithm or data structure to produce a function T(n) measuring the complexity
  - **order of magnitude (asymptotic) analysis**: analysis of the function T(n) to determine the general complexity category to which it belongs.

- Algorithm analysis requires a set of rules to determine how operations are to be counted.
- There is no generally accepted set of rules for algorithm analysis.
- In some cases, an exact count of operations is desired; in other cases, a general approximation is sufficient.

Complexity Analysis

Running time of an algorithm

- Depends upon
  - Input Size
  - Nature of Input
- Generally time grows with size of input, so running time of an algorithm is usually measured as function of input size.
- Running time is measured in terms of number of steps/primitive operations performed
- Independent from machine, OS
- Running time is measured by number of steps/primitive operations performed
- Steps means elementary operation like

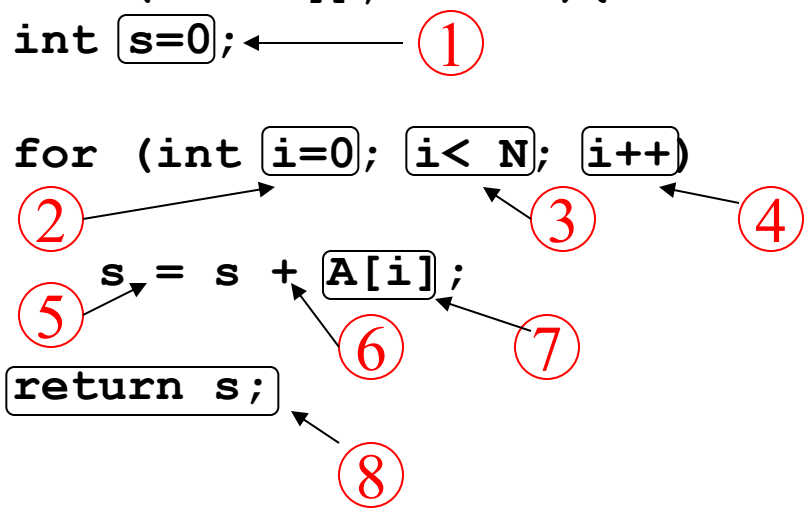
  ,+, *,<, =, A[i] etc
- We will measure number of steps taken in term of size of input

Complexity Analysis

Running time of an algorithm (Example)

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A

int Sum(int A[], int N){
    int s=0;
    for (int i=0; i< N; i++)
        s = s + A[i];
    return s;
}
```



1,2,8: Once

3,4,5,6,7: Once per each iteration of for loop, N iteration

Total: 5N + 3

The *complexity function* of the algorithm is : $f(N) = 5N + 3$

Complexity Analysis

Simple Example (3) Growth of 5n+3

Estimated running time for different values of N:

N = 10	=> 53 steps
N = 100	=> 503 steps
N = 1,000	=> 5003 steps
N = 1,000,000	=> 5,000,003 steps

- As N grows, the number of steps grow in *linear* proportion to N for this function *"Sum"*

- What about the +3 and 5 in 5N+3?
  - As N gets large, the +3 becomes insignificant
  - 5 is inaccurate, as different operations require varying amounts of time and also does not have any significant importance

  - What is fundamental is that the time is *linear* in N.

Asymptotic Complexity: As N gets large, concentrate on the highest order term:
- Drop lower order terms such as +3
- Drop the constant coefficient of the highest order term i.e. N
- The 5N+3 time bound is said to "grow asymptotically" like N
- This gives us an approximation of the complexity of the algorithm
- Ignores lots of (machine dependent) details, concentrate on the bigger picture

Complexity Analysis

COMPARING FUNCTIONS: ASYMPTOTIC NOTATION

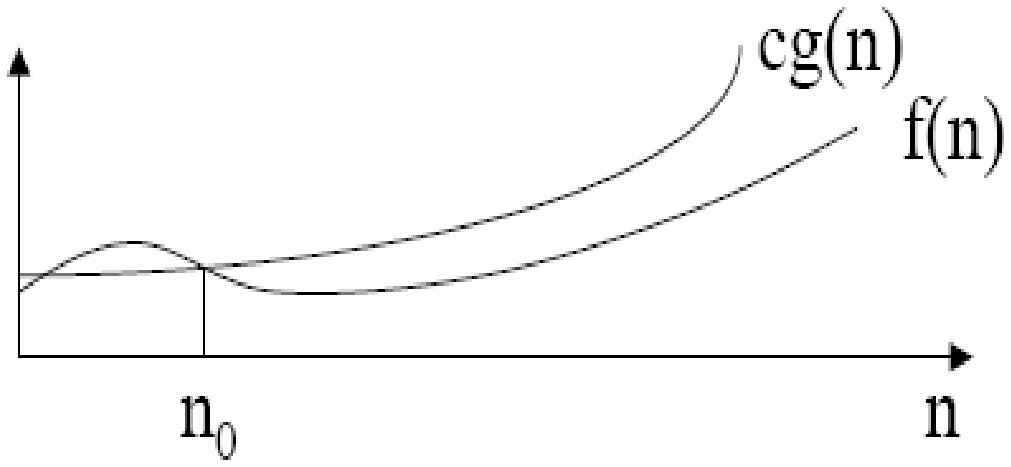
- Big Oh Notation: Upper bound
- Omega Notation: Lower bound
- Theta Notation: Tighter bound

Complexity Analysis

Big Oh Notation

- Big oh(O) notation is used to represent upperbound of algorithm runtime.
- Let f(n) and g(n) are two non-negative functions. The function f(n) = O(g(n)) if and only if there exists positive constants c and n_0 such that f(n)≤c*g(n) for all n , n ≥ n_0.
- In Other words: If f(n) and g(n) are two complexity functions, we say f(n) = O(g(n)) *(read "f(n) is order g(n)", or "f(n) is big-O of g(n)")* if there are constants c and n_0 such that for n ≥ n_0,

  f(n) ≤ c * g(n) for all sufficiently large n.



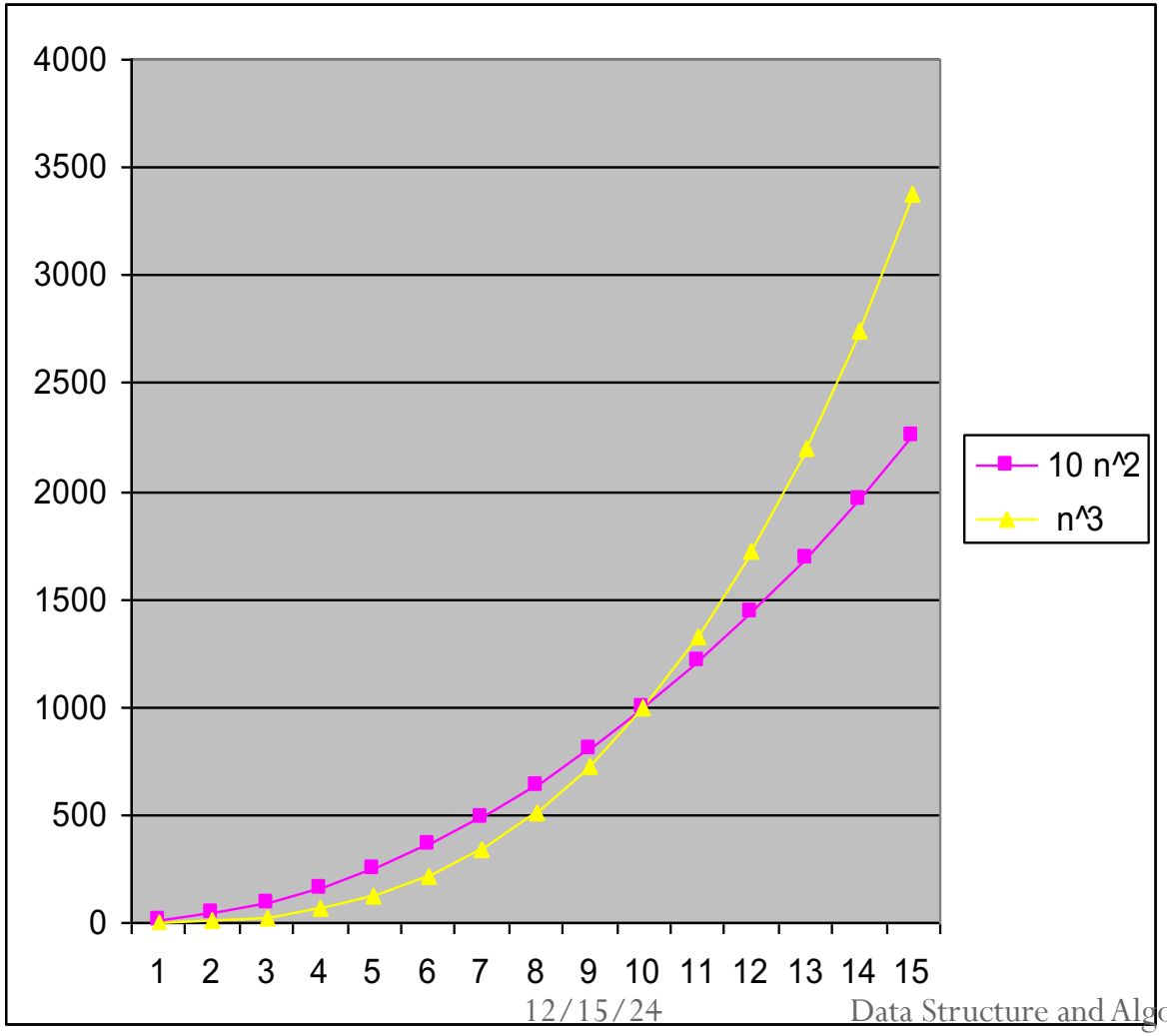
- Function cg(n) always dominates f(n) to the right of n_0

Complexity Analysis

Comparing Functions

Which function is better?

$10n^2$ Vs n^3



Function Definitions:

- $f_1(n)=10n^2$: A quadratic function.
- $f_2(n)=n^3$: A cubic function.

Comparing Growth Rates:

- When n is small, $10n^2$ may be larger than n^3 due to the constant factor 10.
- As n increases, n^3 grows faster than $10n^2$. This is because the cubic term n^3 dominates the quadratic term $10n^2$ for large n.

Cross-over Point: The two functions are equal when $10n^2 = n^3$.

Solving for n:

$n^3 - 10n^2 = 0 \Longrightarrow n^2(n-10)=0$

The non-zero solution is n=10. Thus:

- For $n<10$, $10n^2 > n^3$.
- For $n>10$, $n^3 > 10n^2$.

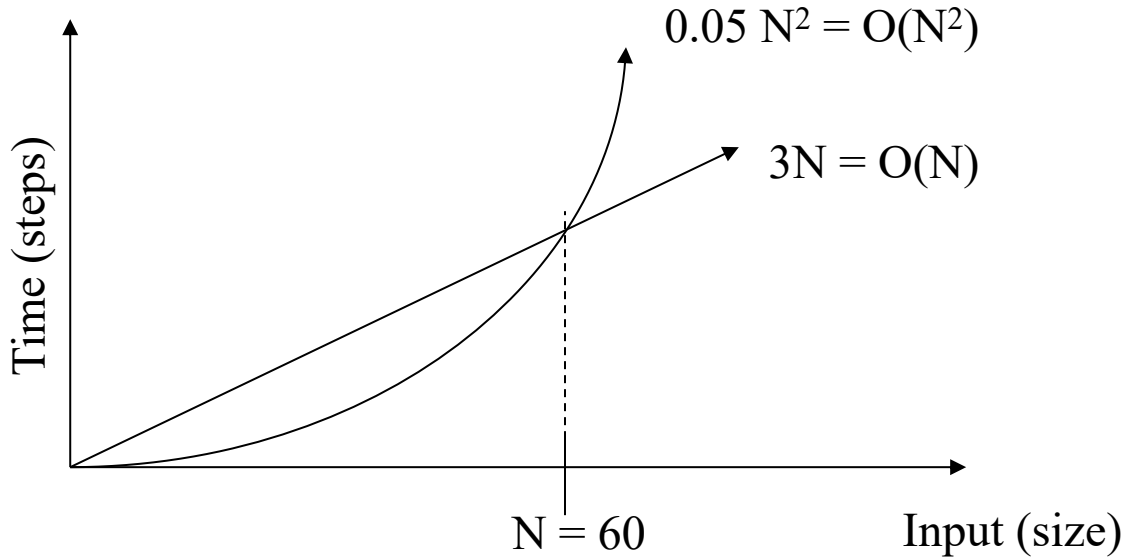
Asymptotic Behavior: In terms of **Big-O notation**:

- $10n^2 = O(n^2)$: Grows quadratically.
- $n^3 = O(n^3)$: Grows cubically.
- As $n \to \infty$, n^3 becomes significantly larger than $10n^2$.

Complexity Analysis

Comparing Functions

- As inputs get larger, any algorithm of a smaller order will be more efficient than an algorithm of a larger order



Even though it is correct to say "7n - 3 is $O(n^3)$", a better statement is "7n - 3 is O(n)", that is, one should make the approximation as tight as possible

Simple Rule:

- Drop lower order terms and constant factors
  - 7n-3 is **O(n)**
  - $8n^2 \log n + 5n^2 + n$ is **$O(n^2 \log n)$**

If f(n)=3n+2 then prove that f(n) = O(n)

Let f(n) =3n+2, c=4, g(n) =n

if n=1, $3n+2 \leq 4n$

$3(1)+2 \leq 4(1)$

$3+2 \leq 4$

$5 \leq 4$ (F)

if n=2, $3n+2 \leq 4n$

$3(2)+2 \leq 4(2)$

$8 \leq 8$ (T)

$3n+2 \leq 4n$ for all $n \geq 2$

This is in the form of $f(n) \leq c*g(n)$ for all $n \geq n_0$, where c=4, n_0 =2

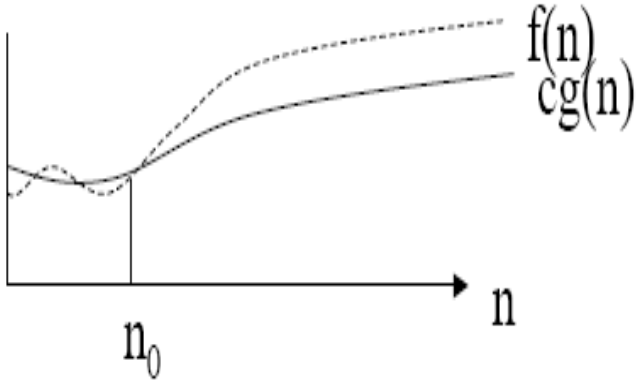
Therefore, f(n) = **O(n)**

Complexity Analysis

Big Omega Notation

- If we wanted to say "running time is at least…" we use Ω
- Big Omega notation, Ω, is used to express the lower bounds on a function.
- If f(n) and g(n) are two complexity functions then we can say:

f(n) is Ω(g(n)) if there exist positive numbers c and n_0 such that 0<=f(n)>=cΩ (n) for all n ≥ n_0



- Example : 3n + 2 = Ω(n)

- In this instance, function cg(n) is dominated by function f(n) to the right of n_0

f(n)=3n+2 then prove that f(n) = Ω(g(n))

Let f(n) =3n+2, c=3, g(n) =n

if n=1, 3n+2 ≥ 3n

3(1)+2 ≥ 3(1)

5 ≥ 3 (T)

3n+2 ≥ 4n for all n ≥ 1

This is in the form of f(n) ≥ c*g(n) for all n ≥ n_0,

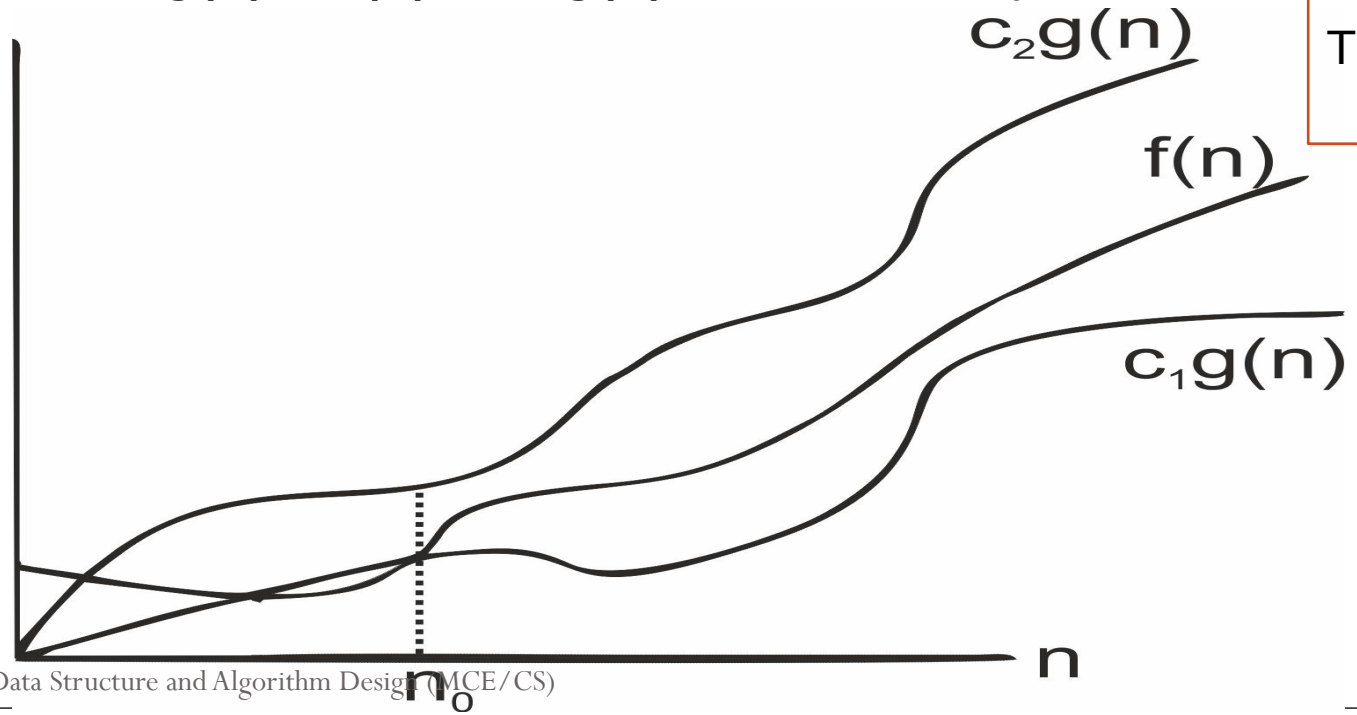
where c=3, n_0 =1

Therefore, f(n) = Ω(n).

Complexity Analysis

Big Theta Notation

- Theta(ɵ) notation is used to represent the running time between upper bound and lower bound.
- If we wish to express tight bounds we use the theta notation, Θ
- Let f(n) and g(n) be two non-negative functions.
- The function f(n) = θ(g(n)) if and only if there exists positive constants c1 , c2 and n_0 such that c_1*g(n) ≤ f(n)≤c_2* g(n) for all n, n≥n_0 .



f(n)=3n+2 then Prove that f(n) = θ(g(n))

Lower bound = 3n+2 ≥ 3n for all n ≥ 1

c_1=3,g(n)=n,n_0=1

Upper Bound = 3n+2 ≤ 4n for all n ≥ 2

c_2=4, g(n)=n , n_0=2

3(n) ≤ 3n+2 ≤ 4(n) for all n, n ≥ 2

This is in the form of c_1*g(n) ≤ f(n) ≤ c_2* g(n) for all n≥n_0 Where c_1=3, c_2=4, g(n)=n, n_0=2

Therefore f(n)= θ(n)

Complexity Analysis

What does this all mean?

- If $f(n) = \Theta(g(n))$ we say that $f(n)$ and $g(n)$ grow at the same rate, asymptotically
- If $f(n) = O(g(n))$ and $f(n) \neq \Omega(g(n))$, then we say that $f(n)$ is asymptotically slower growing than $g(n)$.
- If $f(n) = \Omega(g(n))$ and $f(n) \neq O(g(n))$, then we say that $f(n)$ is asymptotically faster growing than $g(n)$.

Which Notation do we use?

- To express the efficiency of our algorithms which of the three notations should we use?
  - As computer scientist we generally like to express our algorithms as big O since we would like to know the upper bounds of our algorithms.
  - If we know the worse case then we can aim to improve it and/or avoid it.

Complexity Analysis

Performance Classification

f(*n*)	Classification
1	***Constant***: run time is fixed, and does not depend upon n. Most instructions are executed once, or only a few times, regardless of the amount of information being processed
log n	***Logarithmic***: when *n* increases, so does run time, but much slower. Common in programs which solve large problems by transforming them into smaller problems. Exp : binary Search
n	***Linear***: run time varies directly with *n*. Typically, a small amount of processing is done on each element. Exp: Linear Search
n log n	When *n* doubles, run time slightly more than doubles. Common in programs which break a problem down into smaller sub-problems, solves them independently, then combines solutions. Exp: Merge
n^2	***Quadratic***: when *n* doubles, runtime increases fourfold. Practical only for small problems; typically the program processes all pairs of input (e.g. in a double nested loop). Exp: Insertion Search
n^3	***Cubic***: when n doubles, runtime increases eightfold. Exp: Matrix
2^n	***Exponential***: when n doubles, run time squares. This is often the result of a natural, "brute force" solution. Exp: Brute Force. Note: logn, n, nlogn, n^2>> less Input>>Polynomial n^3, 2^n>>high input>> non polynomial

Complexity Analysis

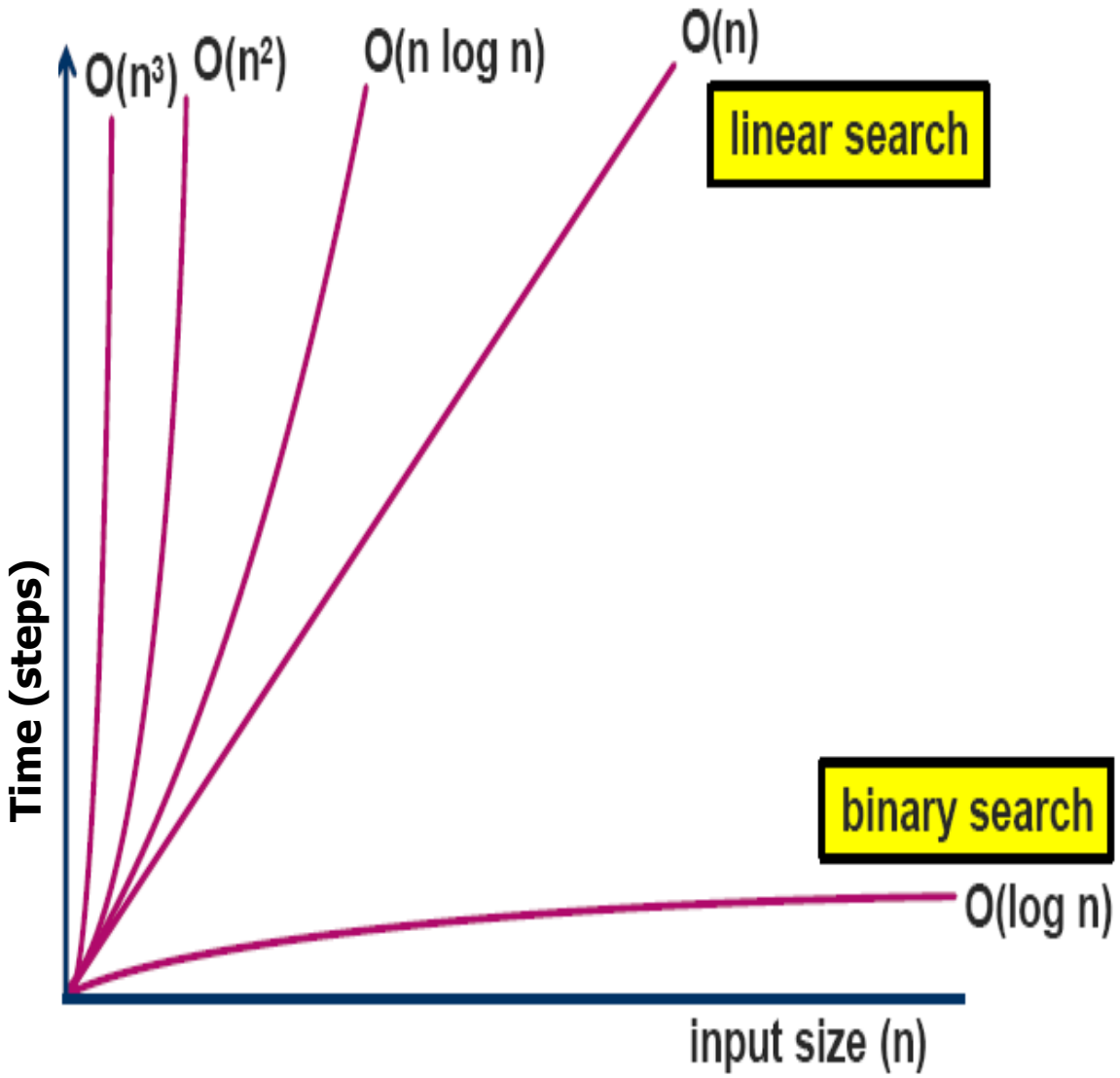
Does Input size matter?

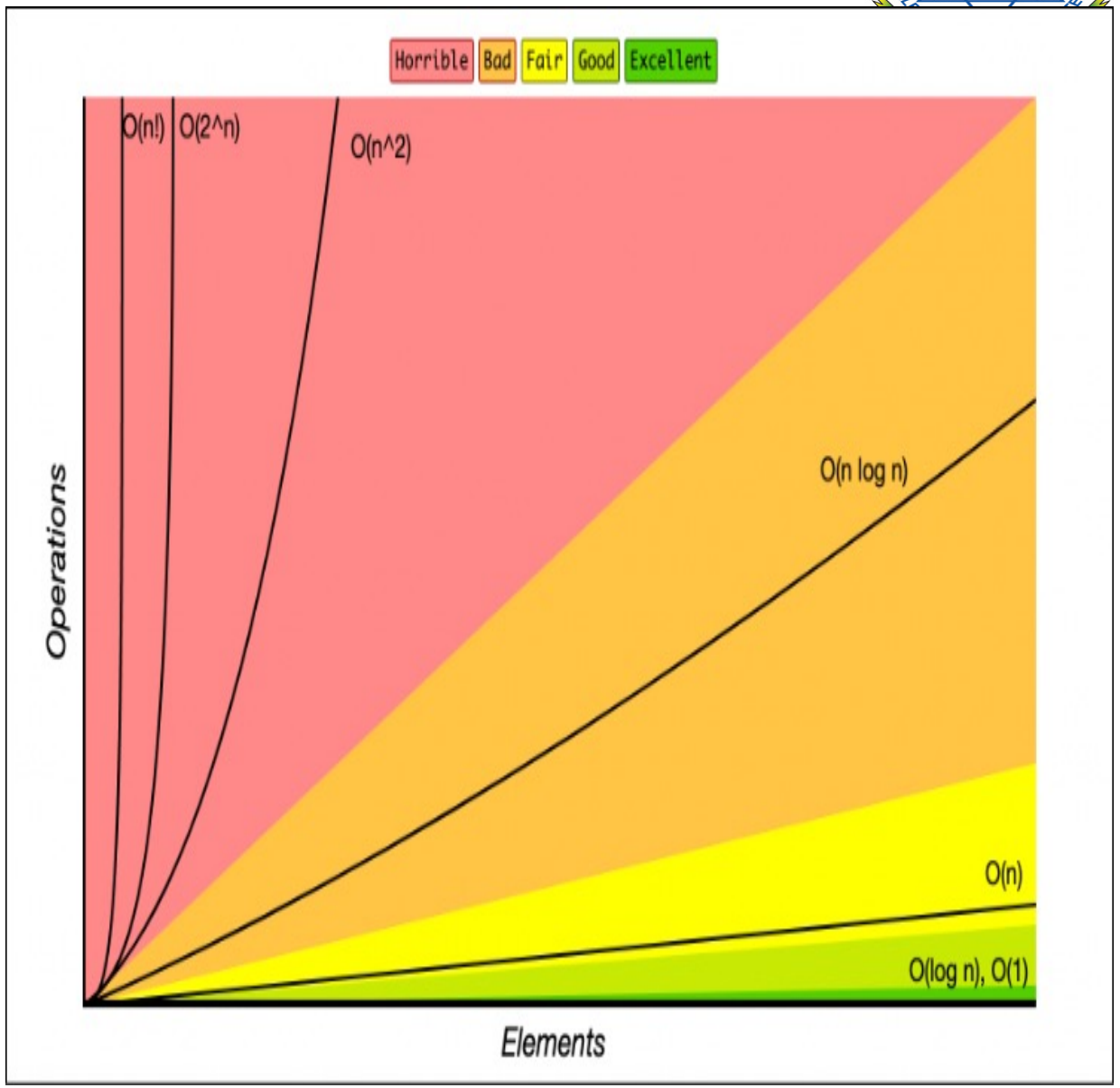
	constant	*logarithmic*	*linear*	*N-log-N*	*quadratic*	*cubic*	*exponential*
n	O(1)	O(log *n*)	O(*n*)	O(*n* log *n*)	O(n^2)	O(n^3)	O(2^n)
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

- Suppose a program has run time O(n!) and the run time for
  n = 10 is 1 second
  For n = 12, the run time is 2 minutes
  For n = 14, the run time is 6 hours
  For n = 16, the run time is 2 months
  For n = 18, the run time is 50 years
  For n = 20, the run time is 200 centuries

Complexity Analysis

Complexity Classes





Complexity Analysis

Standard Analysis Techniques

- Constant time statements
- Analyzing Loops
- Analyzing Nested Loops
- Analyzing Sequence of Statements
- Analyzing Conditional Statements

Complexity Analysis

Constant time statements

- Simplest case: O(1) time statements
- Assignment statements of simple data types
  int x = y;
- Arithmetic operations:
  x = 5 * y + 4 - z;
- Array referencing:
  A[j] = 5;
- Array assignment:
  ∀ j, A[j] = 5;
- Most conditional tests:
  if (x < 12) ...

Analyzing Loops

- Any loop has two parts:
  - How many iterations are performed?
  - How many steps per iteration?

```
int sum = 0,j;
  for (j=0; j < N; j++)
    sum = sum +j;
```

  - Loop executes N times (0..N-1)
  - 4 = O(1) steps per iteration
- Total time is n * O(1) = O(n*1) = **O(n)**

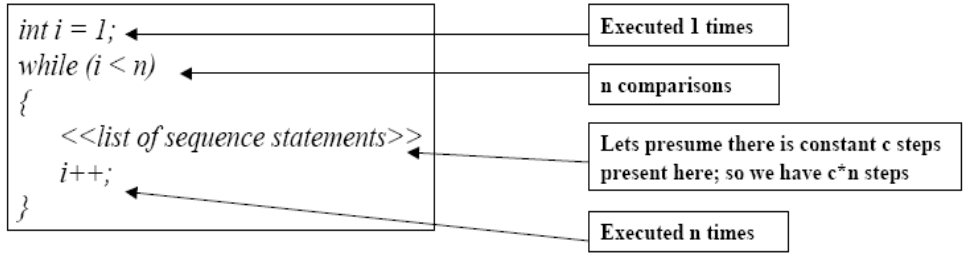
Analyzing Loops

- What about this **for** loop?

```
int sum =0, j;
for (j=0; j < 100; j++)
   sum = sum +j;
```

- Loop executes 100 times
- 4 = O(1) steps per iteration
- Total time is 100 * O(1) = O(100 * 1) = O(100) = **O(1)**
- Note: O(100) is a constant, which simplifies to **O(1)**

Example (have a look at this code segment):



- Efficiency is proportional to the number of iterations.
- Efficiency time function is :
  f(n) = 1 + (n-1) + c*(n-1) +(n-1)
  = (c+2)*(n-1) + 1
  = (c+2)n – (c+2) +1
- Asymptotically, efficiency is : **O(n)**

Complexity Analysis

Analyzing Nested Loops

- Treat just like a single loop and evaluate each level of nesting as needed:

```
int j,k;
for (j=0; j<N; j++)
    for (k=N; k>0; k--)
        sum += k+j;
```

- Start with outer loop:
  - How many iterations? N
  - How much time per iteration? Need to evaluate inner loop
- Inner loop uses O(N) time
- Total time is N * O(N) = O(N*N) = **$O(N^2)$**

- What if the number of iterations of one loop depends on the counter of the other?

```
int j,k;
for (j=0; j < N; j++)
  for (k=0; k < j; k++)
    sum += k+j;
```

- Analyze inner and outer loop together:
- Number of iterations of the inner loop is:
- 0 + 1 + 2 + ... + (N-1) = **$O(N^2)$**

Complexity Analysis

How Did We Get This Answer?

- When doing Big-O analysis, we sometimes have to compute a series like: 1 + 2 + 3 + ... + (n-1) + n
- i.e. Sum of first n numbers. What is the complexity of this?
- Gauss figured out that the sum of the first n numbers is always:

$$\sum_{i=1}^{n} i = \frac{n * (n+1)}{2} = \frac{n^2 + n}{2} = O(n^2)$$

Sequence of Statements

- For a sequence of statements, compute their complexity functions individually and add them up

```
for (j=0; j < N; j++)
   for (k =0; k < j; k++)
      sum = sum + j*k;
for (l=0; l < N; l++)
   sum = sum -l;
System.out.print("sum is now"+sum);
```

$O(N^2)$

$O(N)$

$O(1)$

- Total cost is $O(n^2) + O(n) + O(1) =$ **$O(n^2)$**

Complexity Analysis

Deriving A Recurrence Equation

- All algorithms that we have been analyzing have been non recursive
- Example : Recursive power method

```
double power( double x, int n) {
    if ( n == 0)
        return 1.0;                  // base calse
    //else
        return power(x, n-1)*x;      // recursive case
}
```

- If N = 1, then running time T(N) is 2
- However if N ≥ 2, then running time T(N) is the cost of each step taken plus time required to compute power(x,n-1). (i.e. T(N) = 2+T(N-1) for N ≥ 2)
- How do we solve this? One way is to use the iteration method.

- This is sometimes known as "Back Substituting".
- Involves expanding the recurrence in order to see a pattern.
- Solving formula from previous example using the iteration method :
- ***Solution*** : Expand and apply to itself :

  Let $T(1) = n_0 = 2$

  $T(N) = 2 + T(N-1)$

  $= 2 + 2 + T(N-2)$

  $= 2 + 2 + 2 + T(N-3)$

  $= 2 + 2 + 2 + \ldots\ldots + 2 + T(1)$

  $= 2N + 2$ remember that $T(1) = n_0 = 2$ for N = 1
- So T(N) = 2N+2 is **O(N)** for last example.

Complexity Analysis

Conditional Statements

```
if (condition)
    statement;
```

- Evaluate the condition: O(1) or based on its complexity.
- Execute statement: The complexity depends on the operation inside. The overall complexity is the sum of the two.

```
if (x > 0)
    y = x + 1;
```

- Condition evaluation: O(1).
- Statement execution: O(1)
- Total complexity: **O(1)**.

```
if (condition)
    statement1;
else
    statement2;
```

Evaluate the condition: O(1) or based on its complexity.
Execute either statement1 or statement2 (not both).
The total complexity is:
O(max(complexity of statement1,complexity of statement2))
Eg:

```
if (x > 0)
    for (i = 0; i < n; i++) sum += i; // O(n)
else
    y = x - 1;                        // O(1)
```

Condition evaluation: O(1)
Branch complexity: O(n) (dominates O(1)).
Total complexity: **O(n)**.

Complexity Analysis

Conditional Statements

```
if (x > 0)
    if (x % 2 == 0)
        for (i = 0; i < n; i++)
        sum += i;   // O(n)
    else
        sum = 1;   // O(1)
else
    y = x - 1;     // O(1)
```

- Condition evaluations: O(1) for each condition.
- Branch complexities: O(n),O(1),O(1).
- Total complexity: **O(n)**.

```
if (arr[mid] == key)
    return mid;   // O(1)
else if (arr[mid] < key)
    low = mid + 1;   // O(1)
else
    high = mid - 1;   // O(1)
```

- Condition evaluations: O(1) for comparisons.
- Statement execution: All operations inside each branch are O(1).
- In a loop (e.g., binary search), the complexity depends on the number of iterations: **O(logn)**

Key Points for Analysis

1.Evaluate the **complexity of the condition**.
2.Assess the **complexity of each branch**.
3.The overall complexity is determined by:
- O(condition complexity)+O(max (branch complexities))

Complexity Analysis

Big-O Notation (Common Complexities)

- $T(n)=O(1)$ // constant time
- $T(n)=O(\log n)$ // logarithmic
- $T(n)=O(n)$ // linear
- $T(n)=O(n^2)$ //quadratic
- $T(n)=O(n^3)$ //cubic
- $T(n)=O(n^c)$, $c \geq 1$ // polynomial
- $T(n)=O(\log^c n)$, $c \geq 1$ // polylogarithmic
- $T(n)=O(n \log n)$

Complexity Analysis

Running Time Estimation

- Suppose we have hardware capable of executing **10^6** instructions per second.
- **How long would it take to execute an algorithm whose complexity function was $T(N) = N^2$ on an input of size $N = 10^8$?**
  - *The total number of operations to be performed would be **$T(10^8)$**:*
    - ***$T(10^8) = (10^8)^2 = 10^{16}$***
  - *The total number of seconds required would be given by **$T(10^8)/10^6$** so:*
    - *Running Time = **$10^{16} / 10^6 = 10^{10}$***
  - *The number of seconds/day is 86,400 so this is about **115,740 days** (317 years).*

- **What if we used an algorithm whose complexity function was: $T(N) = N \log N$ on an input of size $N = 10^8$?**
  - *The total number of operations to be performed would be $T(10^8)$:*
    - ***$T(10^8) = (10^8) \log(10^8) \approx 2.66 * 10^9$***
  - *The total number of seconds required would be given by **$T(10^8)/10^6$** so:*
  - *Running Time **$\approx 2.66*10^9 / 10^6 = 2.66*10^3$** (This is about **44.33** minutes)*

Complexity Analysis

Maximum Problem Size

- Another way of looking at this is to ask, what is the largest problem size that can be handled in a given amount of time, given a complexity function for an algorithm and the hardware speed?
- Assuming the same hardware speed as before, what's the largest input size that could be handled in one hour, if the complexity function is once again: **$T(N) = N^2$**
- The total number of seconds required would again be given by **$T(N)/10^6$** so we're asking what is the maximum value of **N** for which: [*$T(N)/10^6$ is used because the hardware speed is specified as 1 million operations per second*]
  - **$T(N)/10^6 \leq 3600$** OR **$N^2 = 3600 * 10^6$**
- This yields **$N \leq 60000$**

- Applying the same logic, with the complexity function: **$T(N) = N \log N$**
- The total number of seconds required would be **$T(N)/10^6$** so we're asking what is the maximum value of **N** for which: **$N \log N \leq 3600 * 10^6$**
- Solving for equality (Newton's Method) yields about **$N \leq 133{,}000{,}000$**

- The first moral is that for large **N**, the complexity function matters.
- The minor first moral is that for large **N**, **Nlog(N)** is a LOT faster than **N^2**.
- The second moral involves applying this logic to the question of hardware speedup

Complexity Analysis

Faster Hardware?

- If we apply the same analysis, assuming that we can now find hardware that is, say, 100 times faster than the previous hardware (so **10^8** operations per second), the results are revealing:

T(N)	time for N = 10^8	max N in 1 hour
N log(N)	.4433 minutes	~10 billion
N^2	3.17 years	~600,000

- Comparing to the earlier results, speeding up the hardware by a factor of 100:
  - reduces time for same sized problem by a factor of 100 in both cases, so the relative advantage of the N log(N) algorithm is retained
  - increases the max problem size by a factor of 10 for the N^2 case, versus a factor of almost 75 for the N log(N) case

End of Chapter 1